Classification

Table of contents

Packages 1
The classification helper 2
Proteins 5
Gene sets 17
Modules 32
Summary 37

How well does a single protein, gene set or module tell the two groups apart? Nothing is fitted. For each feature, the AUC (area under the ROC curve) is the chance that a random person from the first group has a higher value than a random person from the second: 0.5 is a coin toss, 1 perfect separation with the first group higher, 0 perfect separation the other way. The AUC equals the Mann-Whitney statistic W / (n1 × n2) (Bamber 1975; Hanley and McNeil 1982), so its p comes from the Wilcoxon rank-sum test.

This is single-feature discrimination, not a predictive classifier: no AUC here is measured on people held out of the comparison (decisions log, 2026-09-29).

Five tasks compare the groups: the T1, T2 and T3 levels, and the change over training (T2 – T1) and after the bout (T3 – T2). Each runs at three levels: proteins, gene-set scores and module eigengenes.

Packages

pROC supplies the DeLong and bootstrap intervals and the ROC curves; it is called with `::`.

```
library(here)
library(dplyr)
library(tidyr)
library(purrr)
library(tibble)
library(ggplot2)
library(grid)
library(ComplexHeatmap)
source(here("00_Setup", "Shared_functions", "analysis_reporting_helpers.R"))
params$grouping
```

```
[1] "Proteome_T1"
```

```
data_dir <- file.path("03_Group_comparisons", params$grouping, "c_data")
design_file <- file.path(data_dir, "01_design.rds")
d <- readRDS(here(design_file))
d$arms
```

```
[1] "C1" "C2"
```

The classification helper

`classify_level()` runs one level end to end and is called three times below. It takes the feature-by-run matrix, the sample sheet, labels and display order, and writes that level's workbook. Its parts:

- Tasks: each person's value at one time point, or their change between two (people missing either biopsy drop out of that task), split by group.
- Per feature and task, `test_feature_discrimination()` (shared helper) returns the AUC, DeLong's 95% interval (pROC; it runs narrow at this n, and collapses at perfect separation, so none is given there), the Wilcoxon p and the n per group. A feature needs three observations per group.
- BH and Storey's q run within task, and within collection for gene sets, since each collection is its own screen.
- `min_p` is the smallest p the rank-sum test can reach with these group sizes, 2 / choose(n1 + n2, n1); with 8 against 8 that is about 0.00016.
- The strongest p < 0.05 features per task (at most 12 each) are drawn as ROC curves, each with a stratified bootstrap 95% interval (2,000 resamples, seeded).

The share of features at p < 0.05 summarises multiple testing; it does not measure how well the groups separate.

```
classify_level <- function(out, name, inputs, paths, values, meta, feature_label, feature_noun,
                           feature_unit, row_groups, row_order, collection_of = NULL,
                           panel_name = \(label, collection) label, observed_share = NULL,
                           groups = d$arms, boot_n = 2000) {
  dir.create(out, recursive = TRUE, showWarnings = FALSE)
  workbook <- paste0(name, ".xlsx")
  by_collection <- if (is.null(collection_of)) character(0) else "collection"
  labelled_heat <- \(...) make_report_heatmap(..., labels = feature_label)
  bh_scope <- if (length(by_collection)) "task and collection," else "task,"
  labelled <- function(x) {
    if (length(by_collection)) x <- mutate(x, collection = collection_of[feature], .after =
feature)
    x <- mutate(x, label = feature_label[feature], .after = all_of(c("feature",
by_collection)))
    if (is.null(observed_share)) {
      return(x)
    }
    mutate(x, observed_share = observed_share[feature], .after = label)
  }

  arm_of <- set_names(as.character(meta$arm), meta$subject)
  by_subject <- function(timepoint) {
    rows <- filter(meta, timepoint == .env$timepoint, arm %in% groups)
    set_names(as.data.frame(values[, rows$sample_id, drop = FALSE]), rows$subject) |>
as.matrix()
  }
  change <- function(from, to) {
    before <- by_subject(from)
    after <- by_subject(to)
    both <- intersect(colnames(before), colnames(after))
    after[, both, drop = FALSE] - before[, both, drop = FALSE]
  }
  split_groups <- function(level, label) {
```

```
    first <- arm_of[colnames(level)] == groups[1]
    list(
      positive = level[, first, drop = FALSE], negative = level[, !first, drop = FALSE],
      label = label
    )
  }
  versus <- paste(groups, collapse = "v")
  label_versus <- paste(groups, collapse = " vs ")
  tasks <- set_names(list(
    split_groups(by_subject("T1"), paste(label_versus, "at T1")),
    split_groups(by_subject("T2"), paste(label_versus, "at T2")),
    split_groups(by_subject("T3"), paste(label_versus, "at T3")),
    split_groups(change("T1", "T2"), paste0(label_versus, ", training change")),
    split_groups(change("T2", "T3"), paste0(label_versus, ", acute change"))
  ), paste0(c("Baseline_", "Trained_", "Acute_", "Training_change_", "Acute_change_"), versus))

  auc <- imap(tasks, \(spec, name) {
    fits <- vapply(rownames(values), \(id) test_feature_discrimination(spec, id), numeric(6))
    tibble(task = name, task_label = spec$label, feature = rownames(values), !!!
as_tibble(t(fits)))
  }) |>
    list_rbind() |>
    labelled() |>
    mutate(fdr = p.adjust(p, "BH"), qvalue = storey_q(p), .by = c(task, all_of(by_collection)))

  counts <- auc |>
    filter(!is.na(p)) |>
    summarise(
      n_positive = max(n_positive), n_negative = max(n_negative),
      n_tested = n(), n_nominal = sum(p < 0.05), n_fdr_05 = sum(fdr < 0.05),
      .by = c(task, task_label, all_of(by_collection))
    ) |>
    mutate(min_p = 2 / choose(n_positive + n_negative, n_positive), .after = n_negative)

  task_labels <- map_chr(tasks, "label")
  overviews <- imap(row_groups, \(rows, group) {
    task_grid <- \(value) results_to_matrix(auc, rows, names(tasks), "task", value)
    maps <- labelled_heat(
      task_grid(\(x) x$auc), sprintf("AUC, %s", label_versus), c(0, 0.5, 1), "AUC",
      column_labels = task_labels
    ) +
      labelled_heat(task_grid(\(x) status_of(x$p, x$fdr)), "Wilcoxon p and BH q",
        row_names = TRUE, column_labels = task_labels
      )
    heatmap_page(maps, sprintf("Discrimination overview: %s", group), paste(
      sprintf("All %s %s, one row each.", format(length(rows), big.mark = ","), feature_noun),
      row_order, "Grey: not tested."
    ))
  })

  hits <- auc |>
```

```
    filter(p < 0.05) |>
    mutate(task = factor(task, names(tasks))) |>
    arrange(task, p, desc(abs(auc - 0.5)), feature) |>
    paginate_report_features("task")
  set.seed(1)
  boot <- pmap(list(as.character(hits$task), hits$feature), \(task, id) {
    s <- observed_comparison_values(tasks[[task]], id)
    pROC::roc(controls = s$neg, cases = s$pos, direction = "<", quiet = TRUE) |>
      pROC::ci.auc(
        method = "bootstrap", boot.n = boot_n, boot.stratified = TRUE, progress = "none"
      ) |>
      as.numeric()
  })
  hits <- hits |>
    mutate(
      boot_lower = map_dbl(boot, 1), boot_upper = map_dbl(boot, 3),
      name = panel_name(label, pick(everything())[["collection"]]),
      panel = sprintf(
        "%s\nAUC %.2f [%.2f, %.2f]  p %.2g  q %.2g\n%s %d, %s %d", name, auc, boot_lower,
        boot_upper, p, fdr, groups[1], n_positive, groups[2], n_negative
      )
    )
  auc <- left_join(
    auc, select(hits, task, feature, boot_lower, boot_upper) |> mutate(task =
as.character(task)),
    by = c("task", "feature")
  )
  task_page <- function(page_hits) {
    spec <- tasks[[as.character(page_hits$task[1])]]
    info <- filter(counts, task == as.character(page_hits$task[1])) |>
      summarise(across(c(task_label, n_positive, n_negative, min_p), first),
        across(c(n_tested, n_nominal, n_fdr_05), sum),
        .by = task
      )
    curves <- map(page_hits$feature, \(id) {
      s <- observed_comparison_values(spec, id)
      pROC::roc(controls = s$neg, cases = s$pos, direction = "<", quiet = TRUE)
    }) |>
      set_names(page_hits$panel)
    pROC::ggroc(curves, aes = "group", legacy.axes = TRUE, colour = "black") +
      geom_abline(
        aes(intercept = 0, slope = 1),
        data = tibble(name = page_hits$panel), linetype = 2, colour = "grey60"
      ) +
      facet_wrap(~ factor(name, levels = slots(page_hits$panel)),
        nrow = 3, ncol = 4, drop = FALSE
      ) +
      coord_equal() +
      labs(
        title = info$task_label, subtitle = counts_line(info, feature_noun, page_hits),
        x = "1 - specificity", y = "sensitivity"
```

```
      ) +
      page_theme
  }
  pages <- map(split(hits, hits$pdf_page), task_page)

  plot_index <- bind_rows(
    tibble(task = "overview", label = map_chr(overviews, "title"), pdf_page =
seq_along(overviews)),
    transmute(hits,
      task = as.character(task), pick(all_of(by_collection)), feature, label,
      pdf_page = pdf_page + length(overviews), position
    )
  )
  stopifnot(
    nrow(hits) == capped_count(filter(auc, p < 0.05), "task"),
    !anyDuplicated(select(filter(plot_index, !is.na(feature)), -label, -pdf_page, -position)),
    max(plot_index$pdf_page) == length(overviews) + length(pages)
  )
  walk(overviews, \(page) page$draw())
  walk(pages, print)

  write_workbook(
    list(
      counts = counts, auc = arrange(auc, task, pick(all_of(by_collection)), p),
      plot_index = plot_index
    ),
    c(
      "Per task: n per group, smallest attainable p, tested, p < 0.05 and BH < 0.05 counts.",
      sprintf(paste(
        "Per %s and task: AUC with DeLong 95%% interval, Wilcoxon p, BH and Storey q;
bootstrap",
        "95%% interval for the features drawn in the report."
      ), feature_unit),
      "PDF page of each overview, and page and panel position (1 to 12) of every drawn row."
    ),
    file.path(out, workbook),
    inputs, paths
  )
  counts
}
```

Proteins

Rank tests take no covariate, so proteins are read from limpa's blood-corrected matrix. `observed_share` records how much of each protein was measured rather than estimated by limpa. Overview rows follow average-linkage clustering on 1 − Pearson r of limpa's uncorrected matrix, for display only.

```
inputs <- c(
  design = design_file,
  corrected = "01_Processing/03_Quantification/c_data/DAList_corrected.rds",
```

```
  limpa = "01_Processing/03_Quantification/c_data/DAList_limpa.rds"
)
paths <- map_chr(inputs, here)
stopifnot(file.exists(paths))
dal <- subset_for_design(readRDS(paths[["corrected"]]), d)
feature_label <- with(dal$annotation, set_names(gene, uniprot_id))
stopifnot(!anyNA(feature_label), !anyDuplicated(feature_label))
complete <- subset_for_design(readRDS(paths[["limpa"]]), d)$data
protein_counts <- classify_level(
  out = here(data_dir), name = "05_protein_classification",
  inputs = inputs, paths = paths, values = dal$data, observed_share = rowMeans(dal$observed),
  meta = dal$metadata, feature_label = feature_label, feature_noun = "proteins",
  feature_unit = "protein",
  row_groups = list(
    proteins = rownames(complete)[hclust(as.dist(1 - cor(t(complete))), "average")$order]
  ),
  row_order = "Rows: average-linkage clustering of limpa's matrix, for display only."
)
```

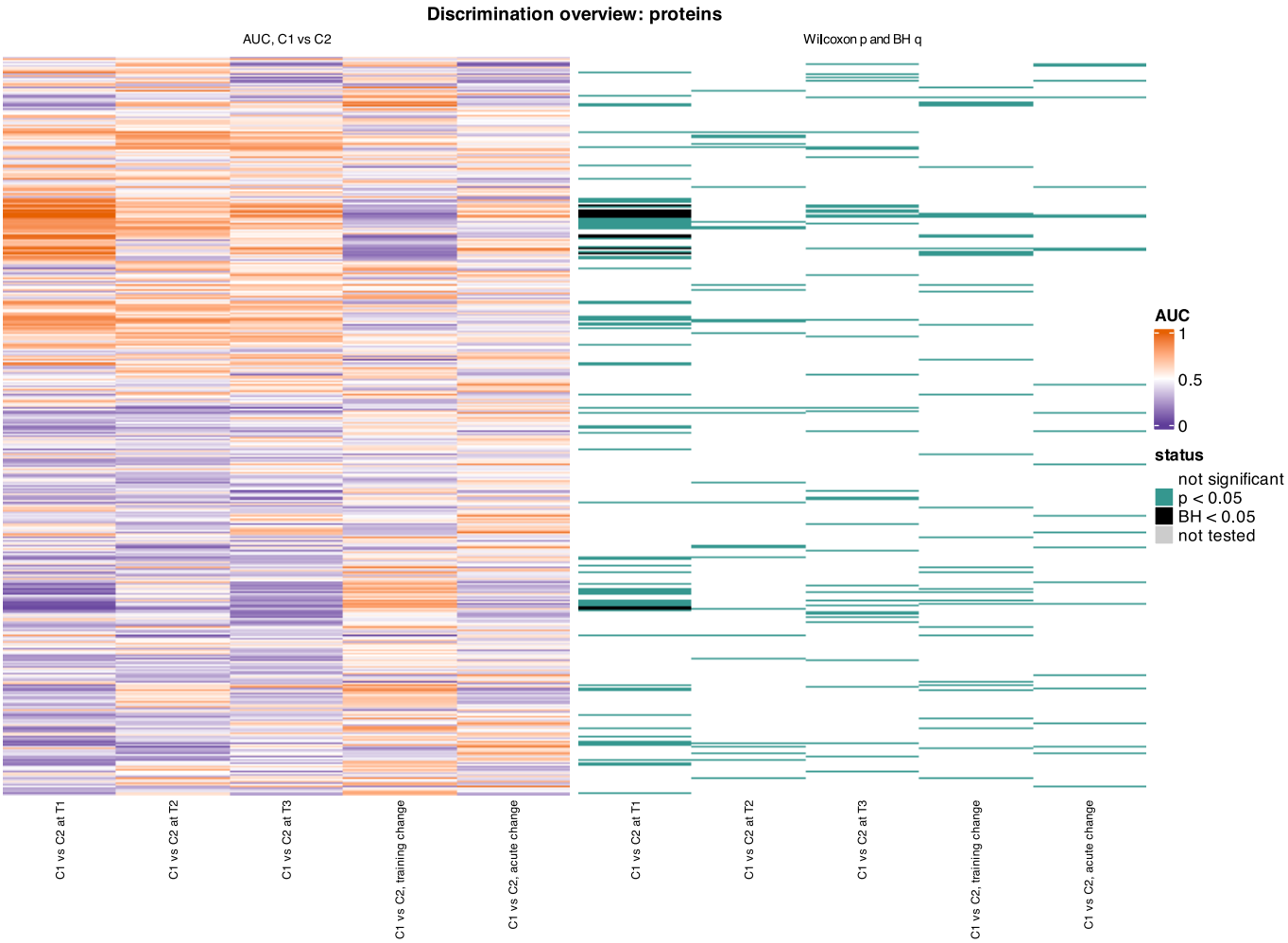


Figure 1: Proteins: AUC and p per task, then the strongest p < 0.05 proteins as ROC curves.

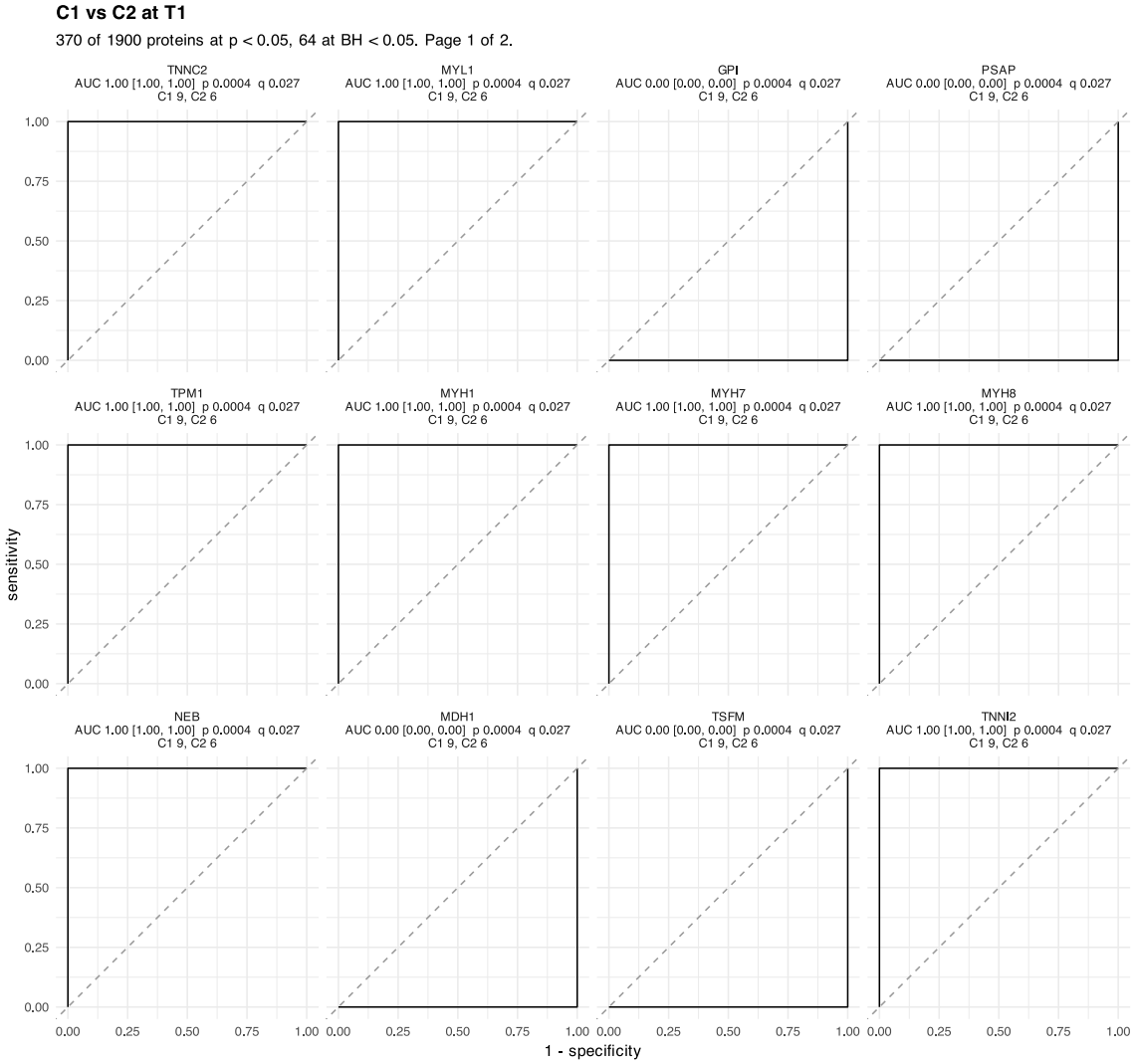


Figure 2: Proteins: AUC and p per task, then the strongest p < 0.05 proteins as ROC curves.

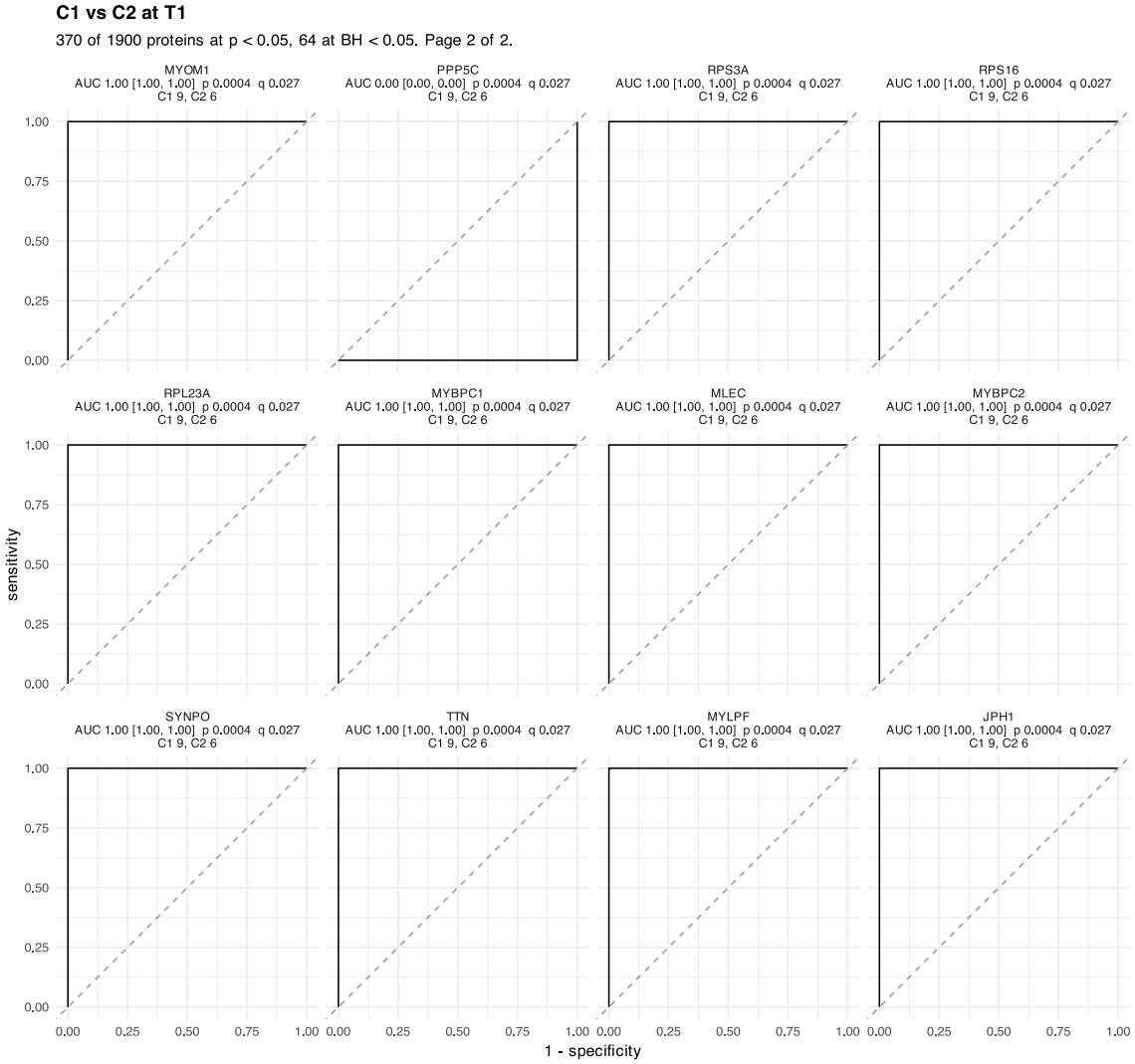


Figure 3: Proteins: AUC and p per task, then the strongest p < 0.05 proteins as ROC curves.

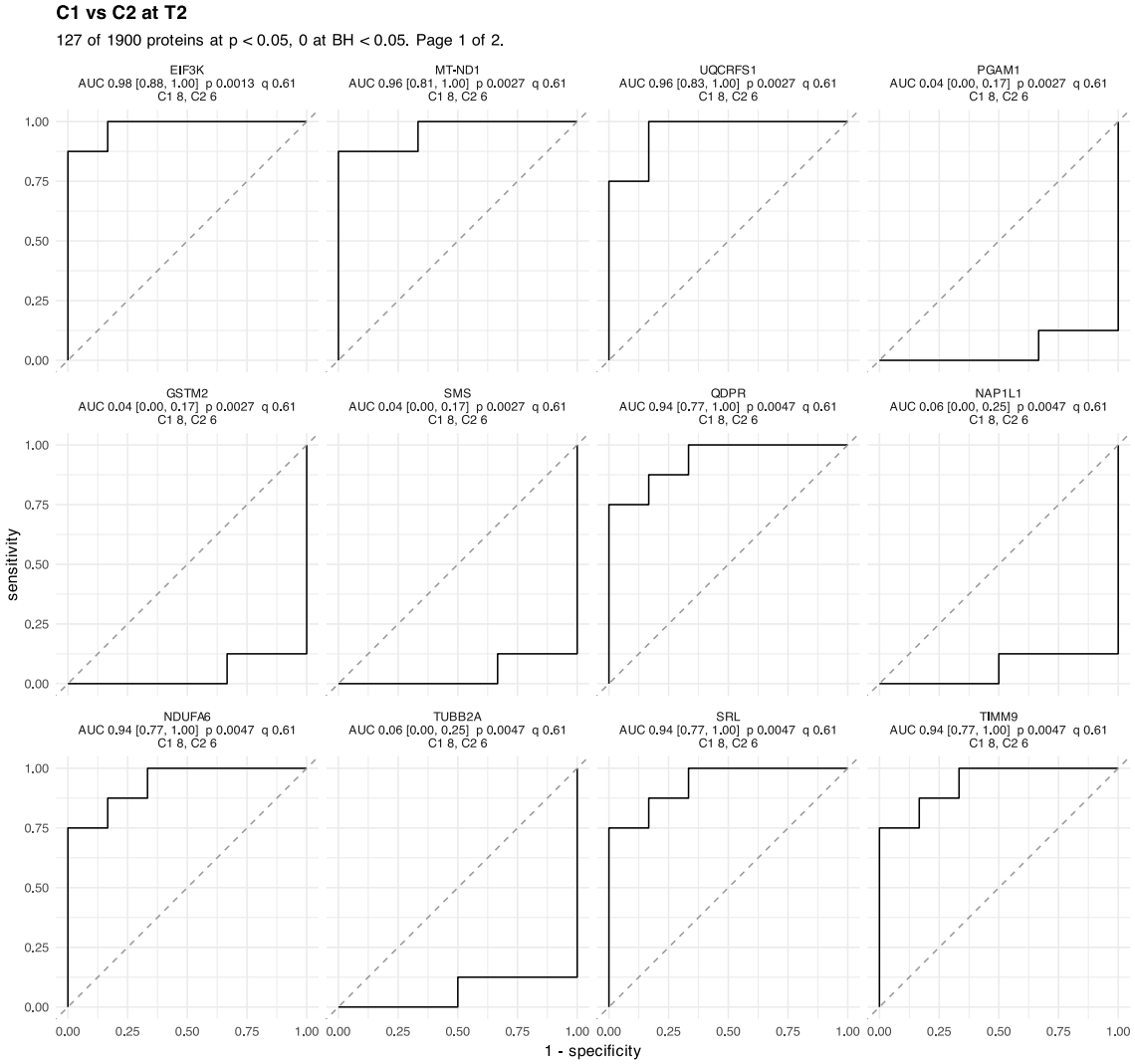


Figure 4: Proteins: AUC and p per task, then the strongest p < 0.05 proteins as ROC curves.

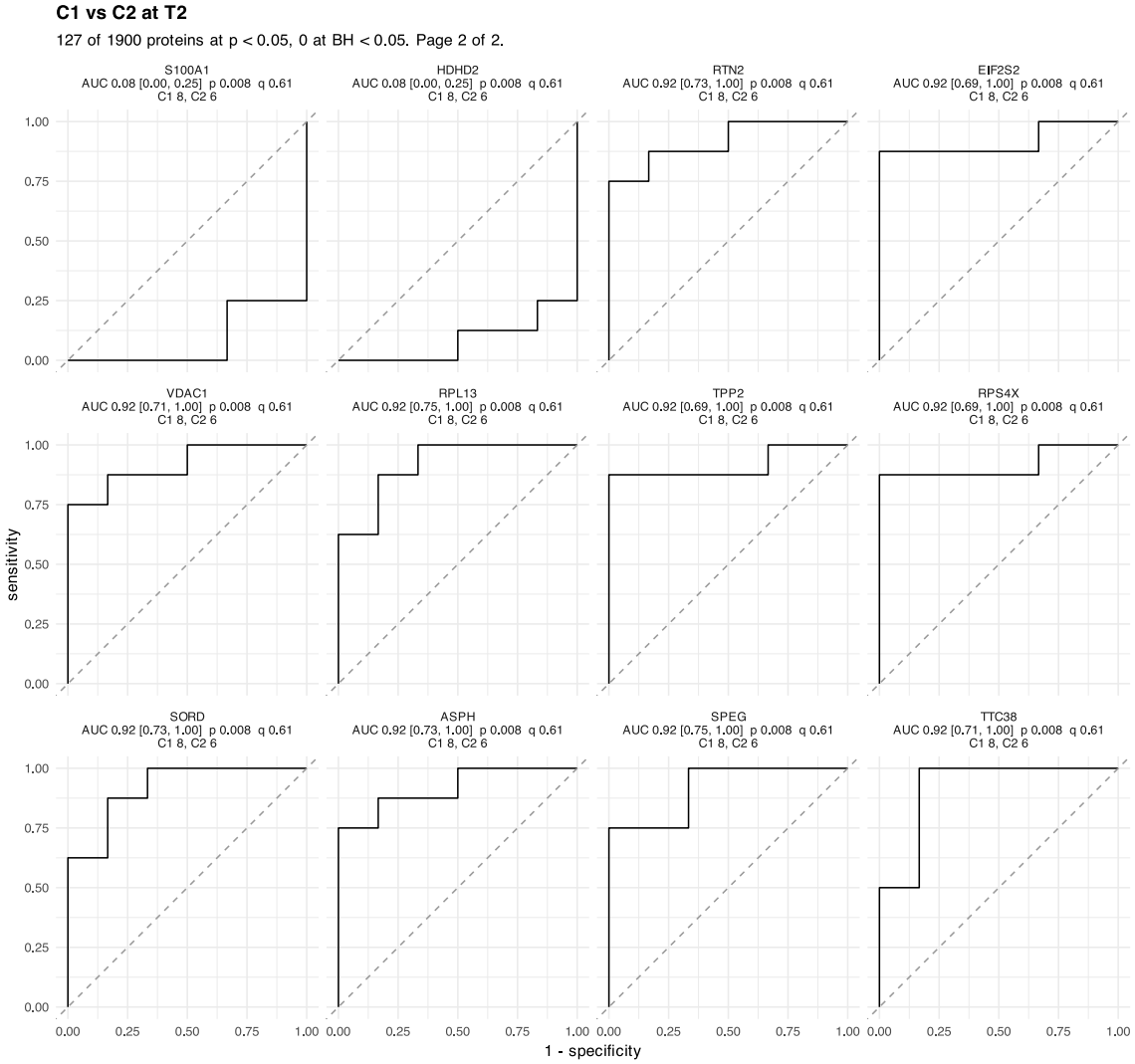


Figure 5: Proteins: AUC and p per task, then the strongest p < 0.05 proteins as ROC curves.

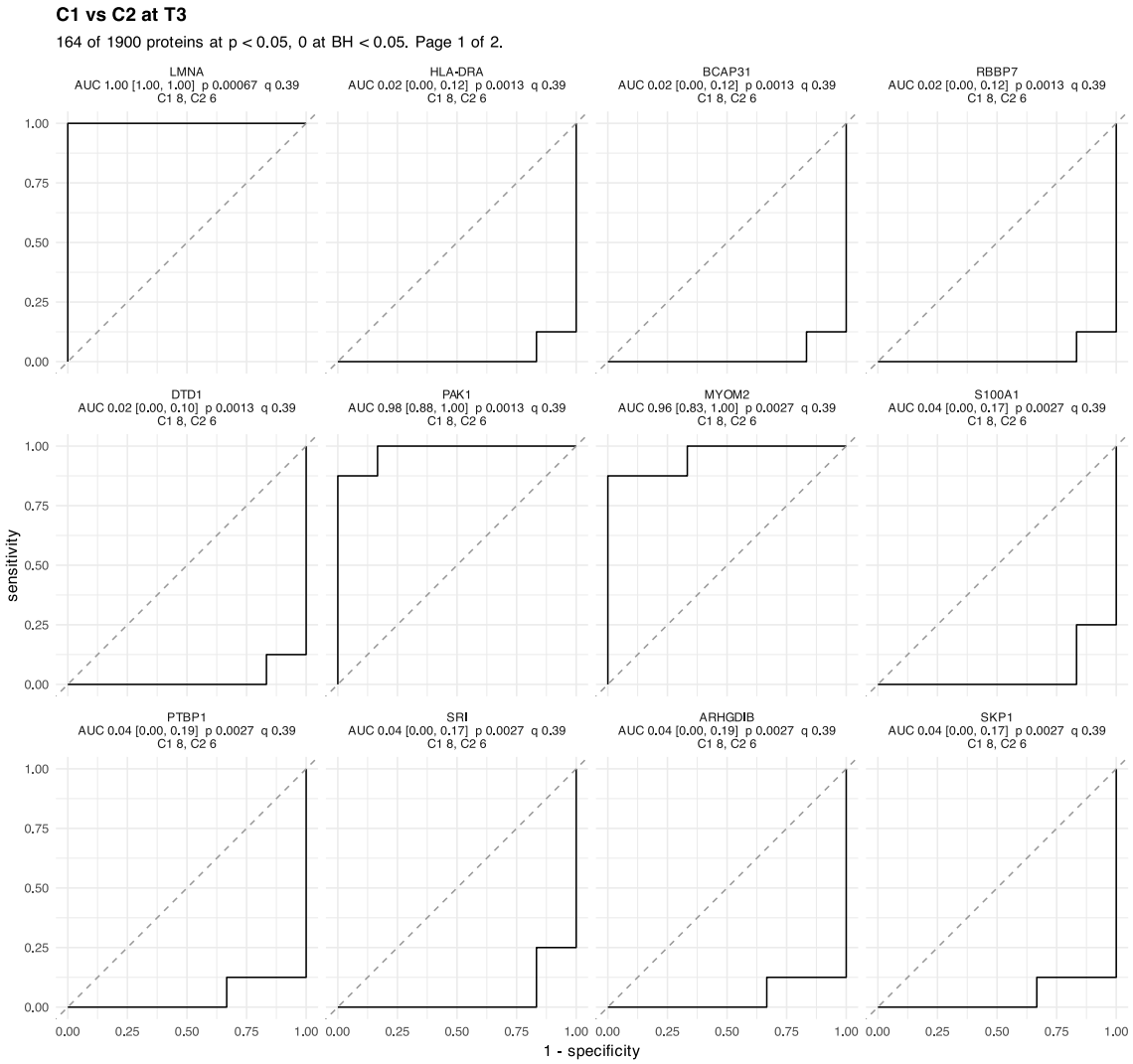


Figure 6: Proteins: AUC and p per task, then the strongest p < 0.05 proteins as ROC curves.

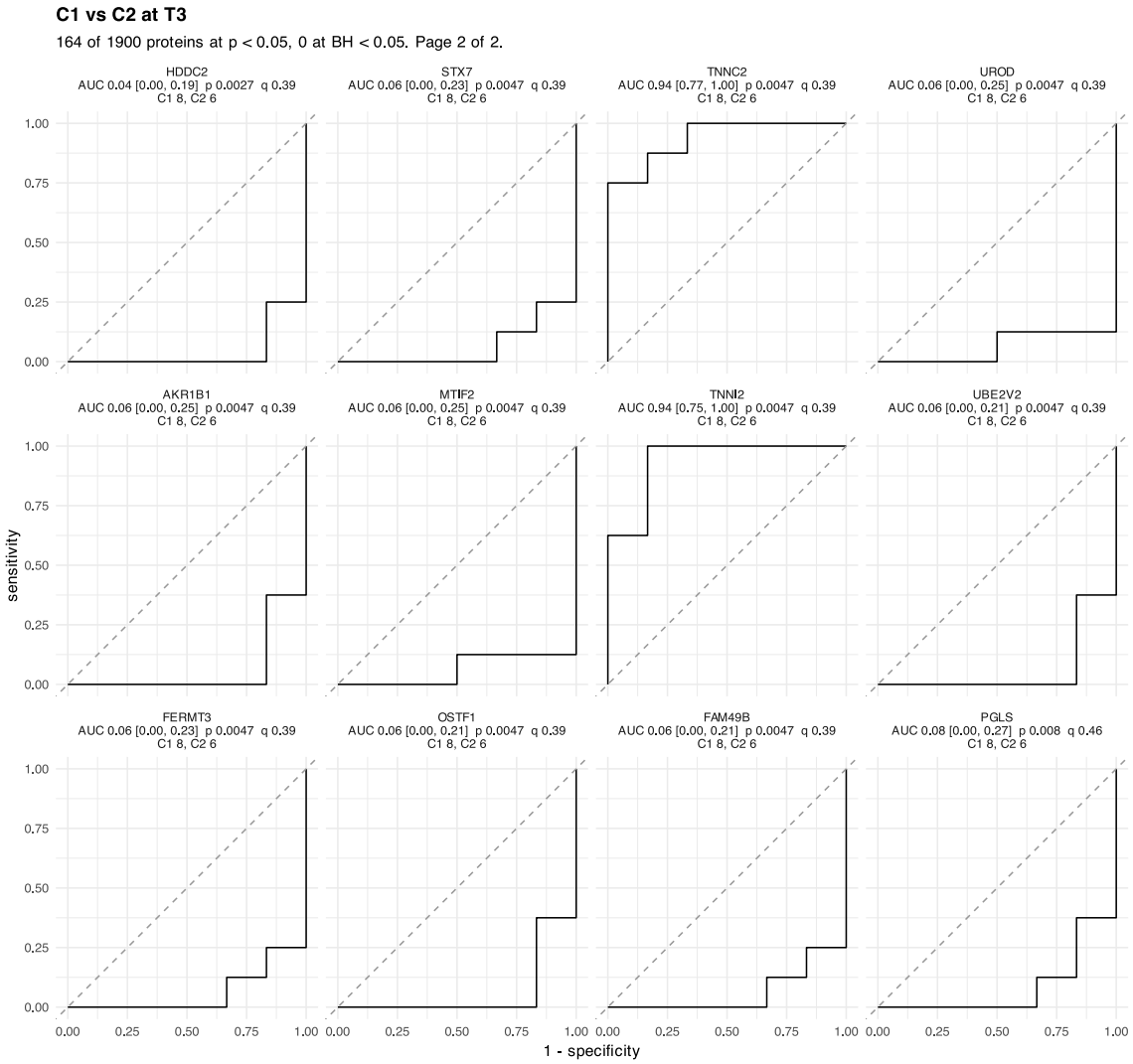


Figure 7: Proteins: AUC and p per task, then the strongest p < 0.05 proteins as ROC curves.

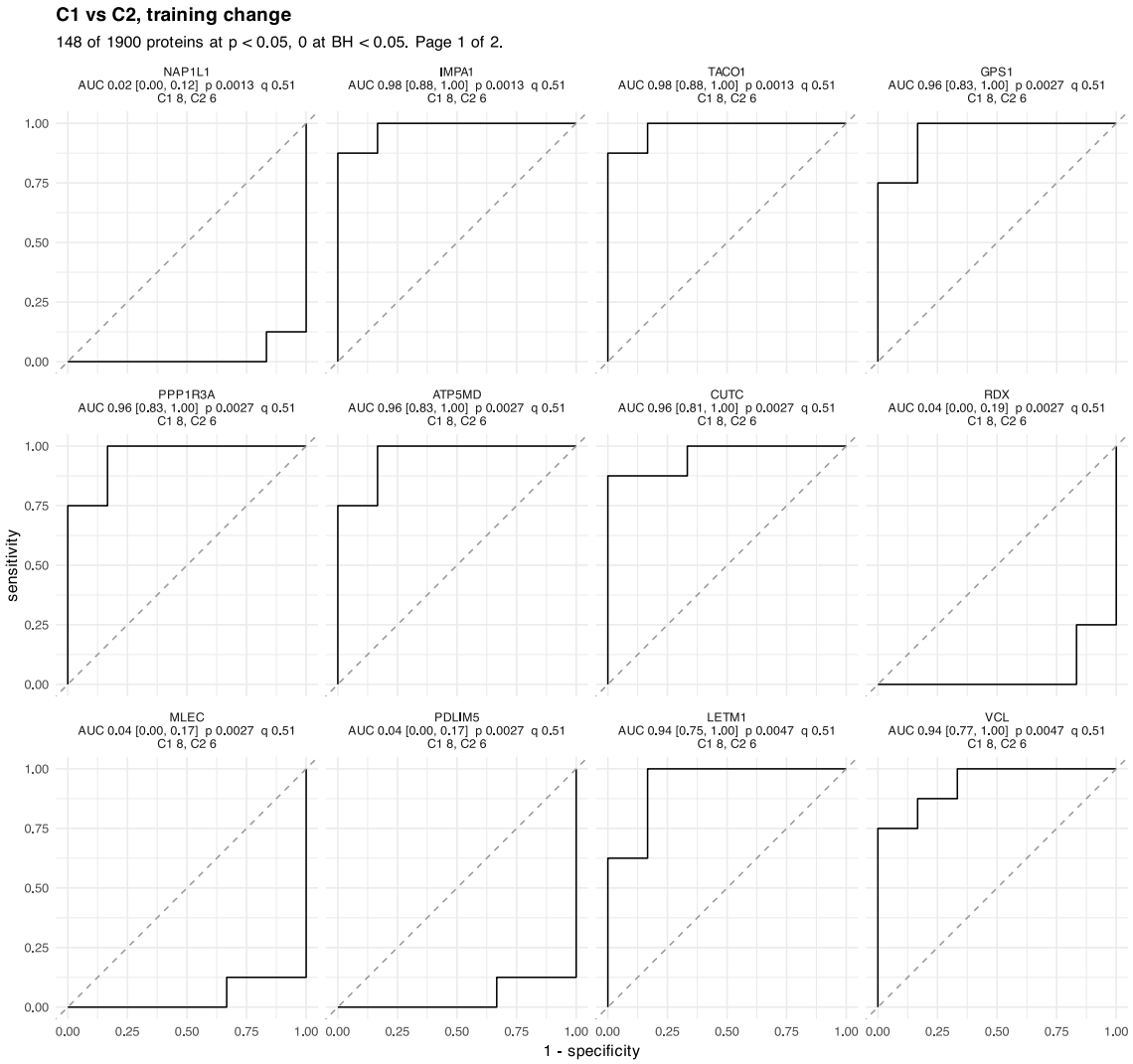


Figure 8: Proteins: AUC and p per task, then the strongest p < 0.05 proteins as ROC curves.

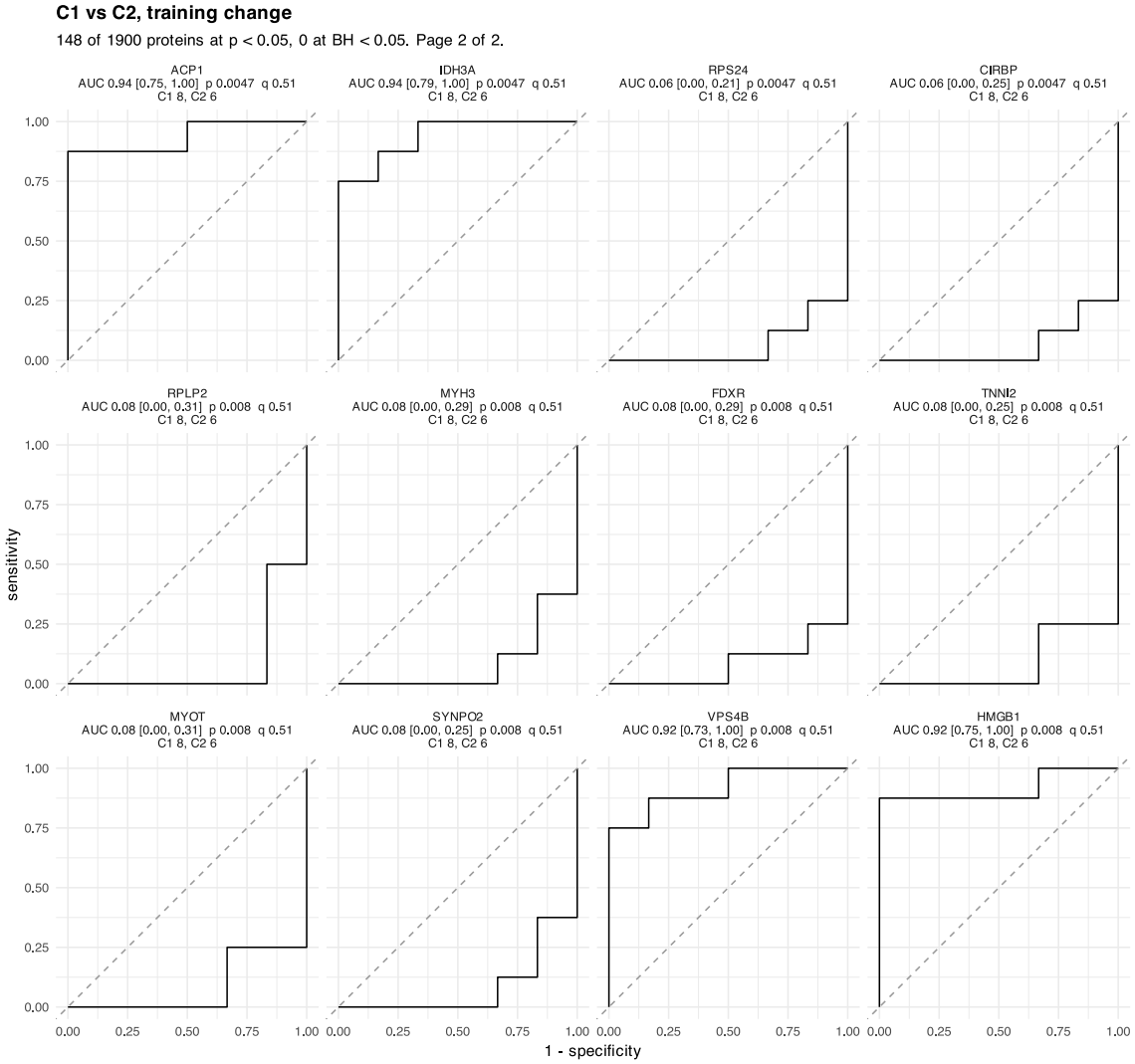


Figure 9: Proteins: AUC and p per task, then the strongest p < 0.05 proteins as ROC curves.

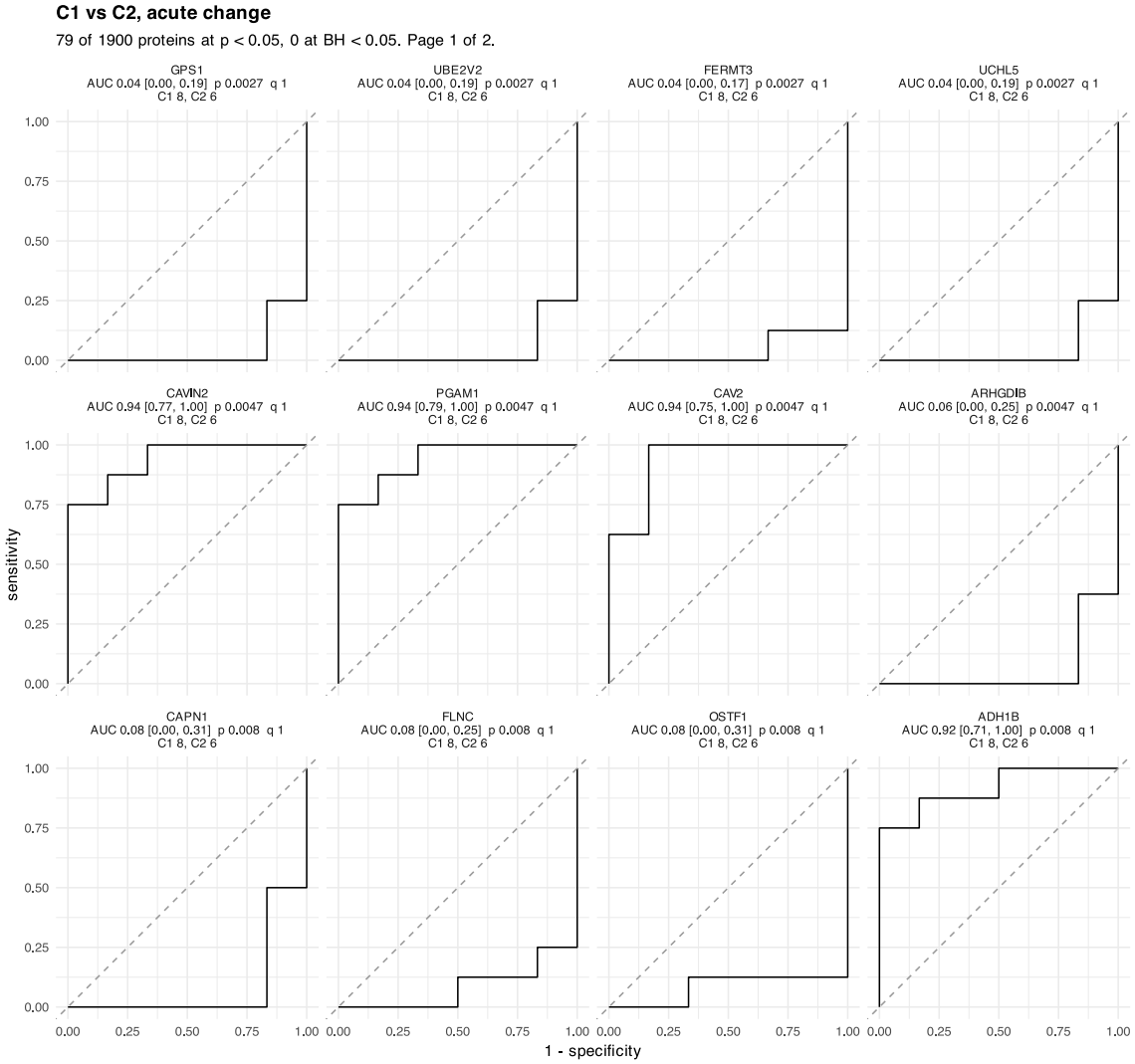


Figure 10: Proteins: AUC and p per task, then the strongest p < 0.05 proteins as ROC curves.

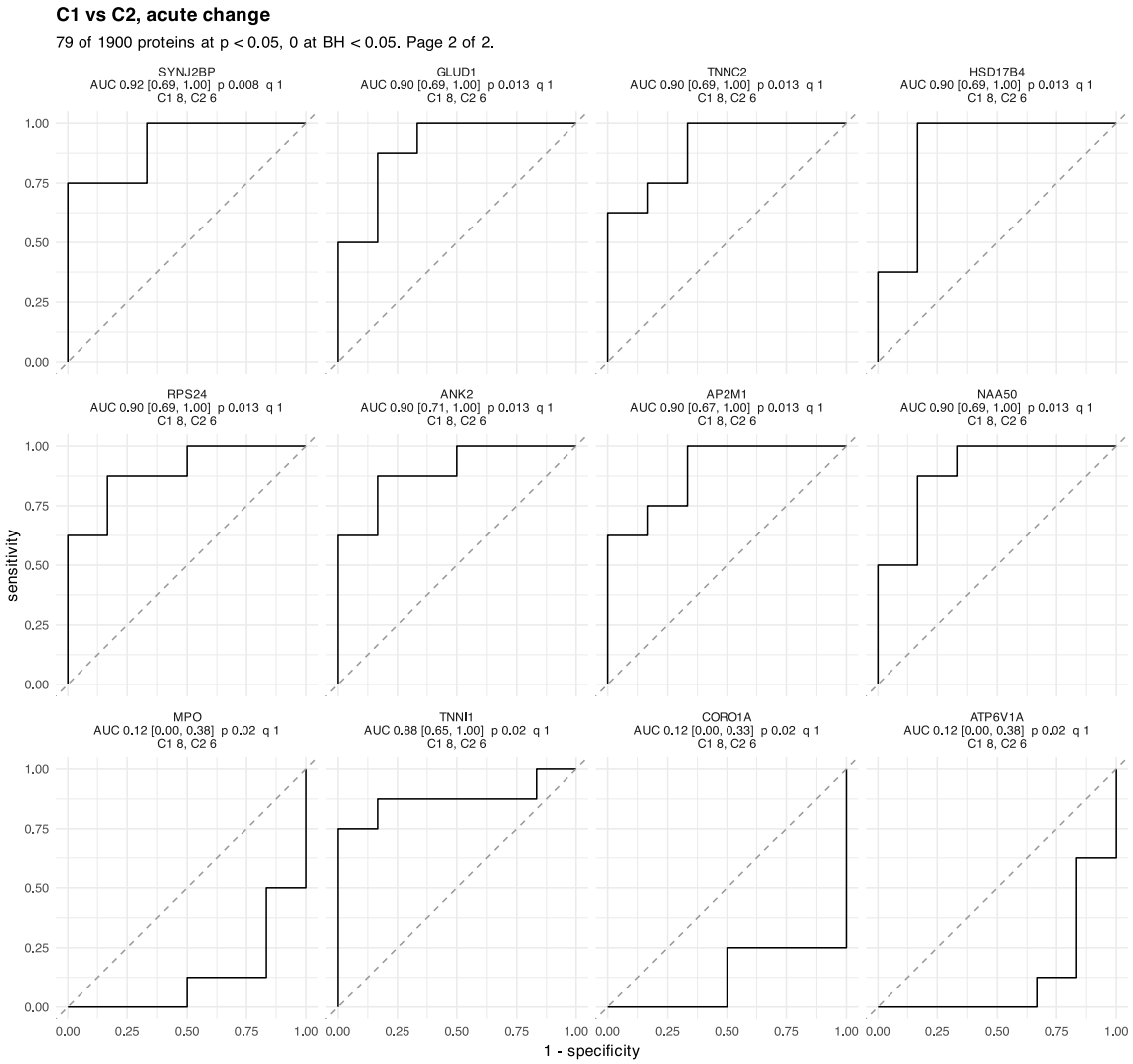


Figure 11: Proteins: AUC and p per task, then the strongest p < 0.05 proteins as ROC curves.

Gene sets

Each set's singscore per run. Each collection is its own screen, so BH runs within task and collection.

```
inputs <- c(
  design = design_file,
  singscore = "01_Processing/05_Set_Scores/c_data/singscore.rds",
  gene_sets = "01_Processing/04_Gene_Sets/c_data/gene_sets.rds",
  proteins = "01_Processing/03_Quantification/c_data/DAList_corrected.rds"
)
paths <- map_chr(inputs, here)
stopifnot(file.exists(paths))
catalog <- readRDS(paths[["gene_sets"]])$set_catalog |> filter(qualifies)
values <- subset_for_design(readRDS(paths[["singscore"]]), d)$scores[catalog$set_id, ]
meta <- subset_for_design(readRDS(paths[["proteins"]]), d)$metadata
stopifnot(identical(colnames(values), meta$sample_id))
```

```
set_counts <- classify_level(
  out = here(data_dir), name = "05_set_classification",
  inputs = inputs, paths = paths, values = values, meta = meta,
  feature_label = with(catalog, set_names(enrichVolcano::ev_clean_label(pathway), set_id)),
  feature_noun = "sets", feature_unit = "set",
  row_groups = split(catalog$set_id, factor(catalog$collection, unique(catalog$collection))) |>
    map(\(ids) ids[hclust(as.dist(1 - cor(t(values[ids, ]))), "average")$order]),
  row_order = "Rows: average-linkage clustering of singscore within collection, display only.",
  collection_of = with(catalog, set_names(collection, set_id)),
  panel_name = \(label, collection) {
    paste0(collection, ": ", stringr::str_wrap(gsub("\n", " ", label), 50))
  }
)
```

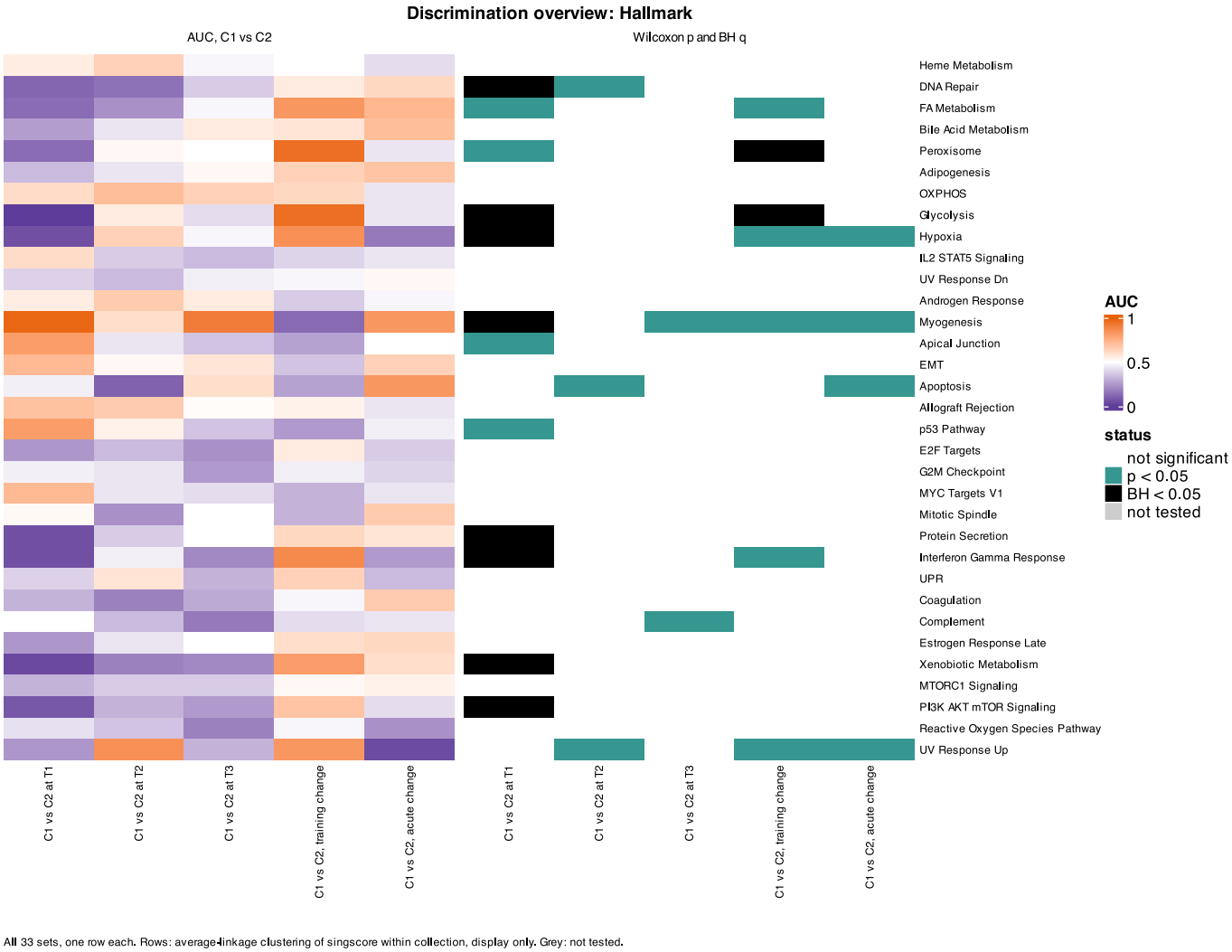


Figure 12: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.



Figure 13: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.



All 62 sets, one row each. Rows: average-linkage clustering of singscore within collection, display only. Grey: not tested.

Figure 14: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

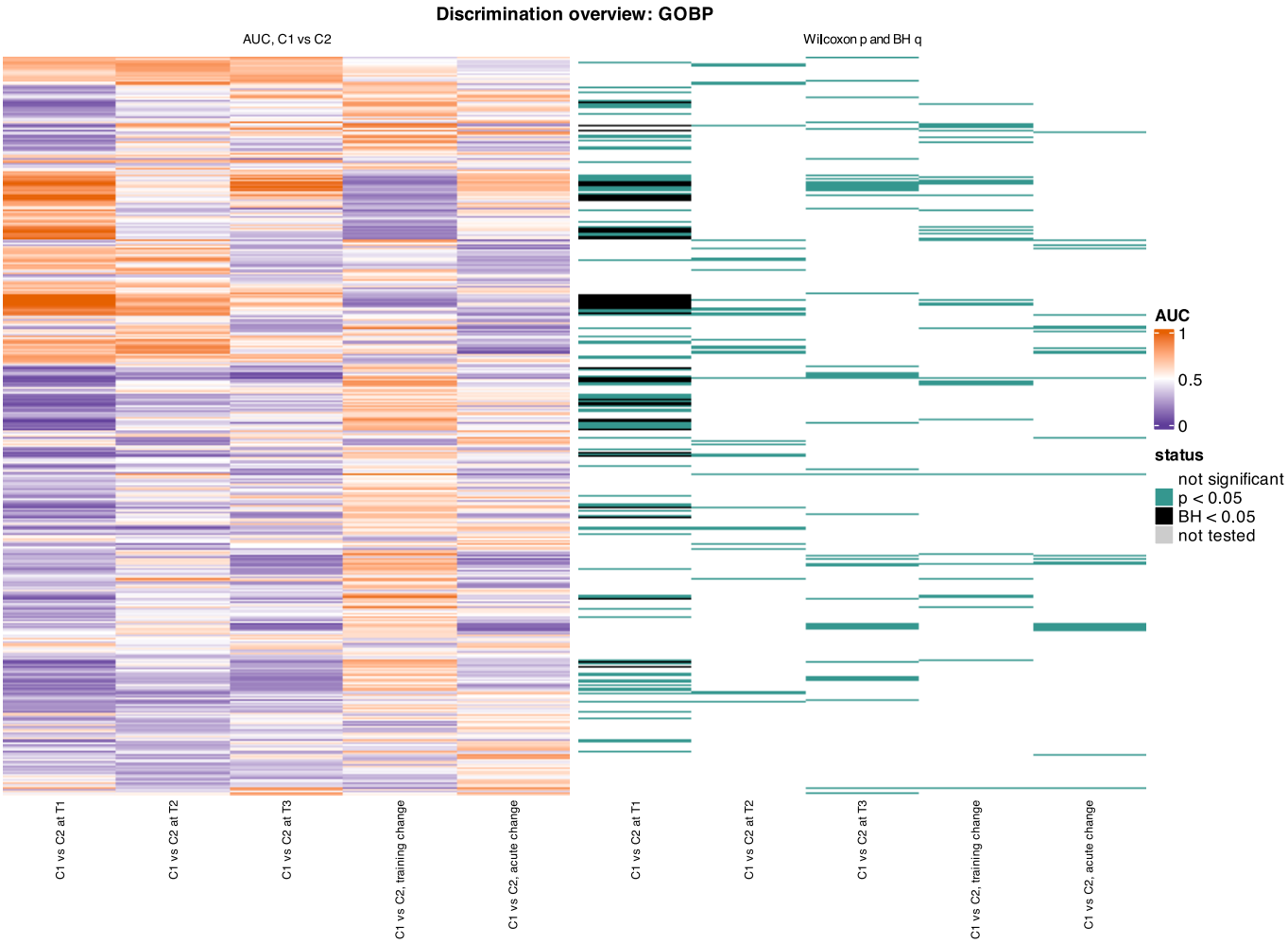
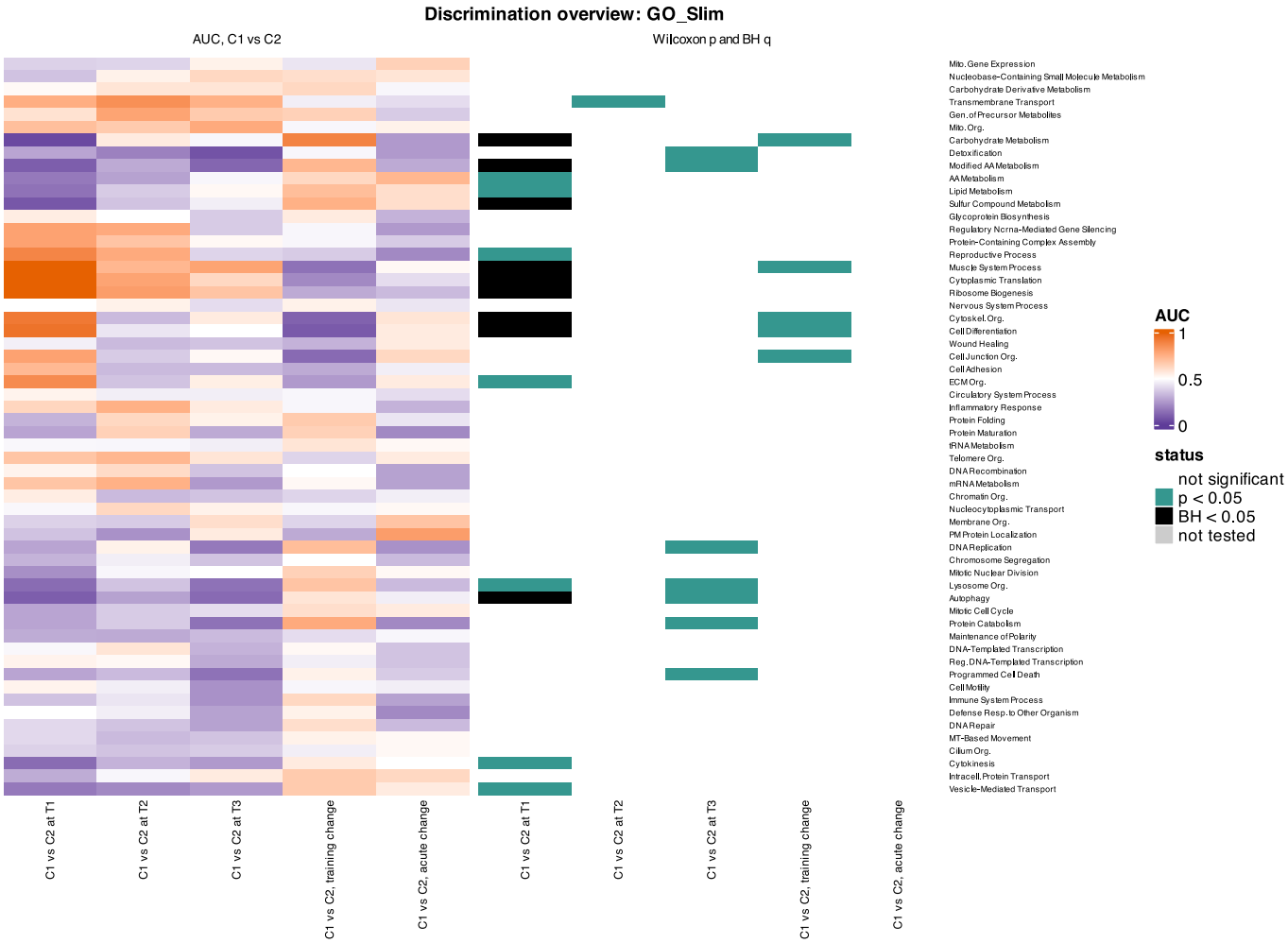


Figure 15: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.



All 58 sets, one row each. Rows: average-linkage clustering of singscore within collection, display only. Grey: not tested.

Figure 16: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

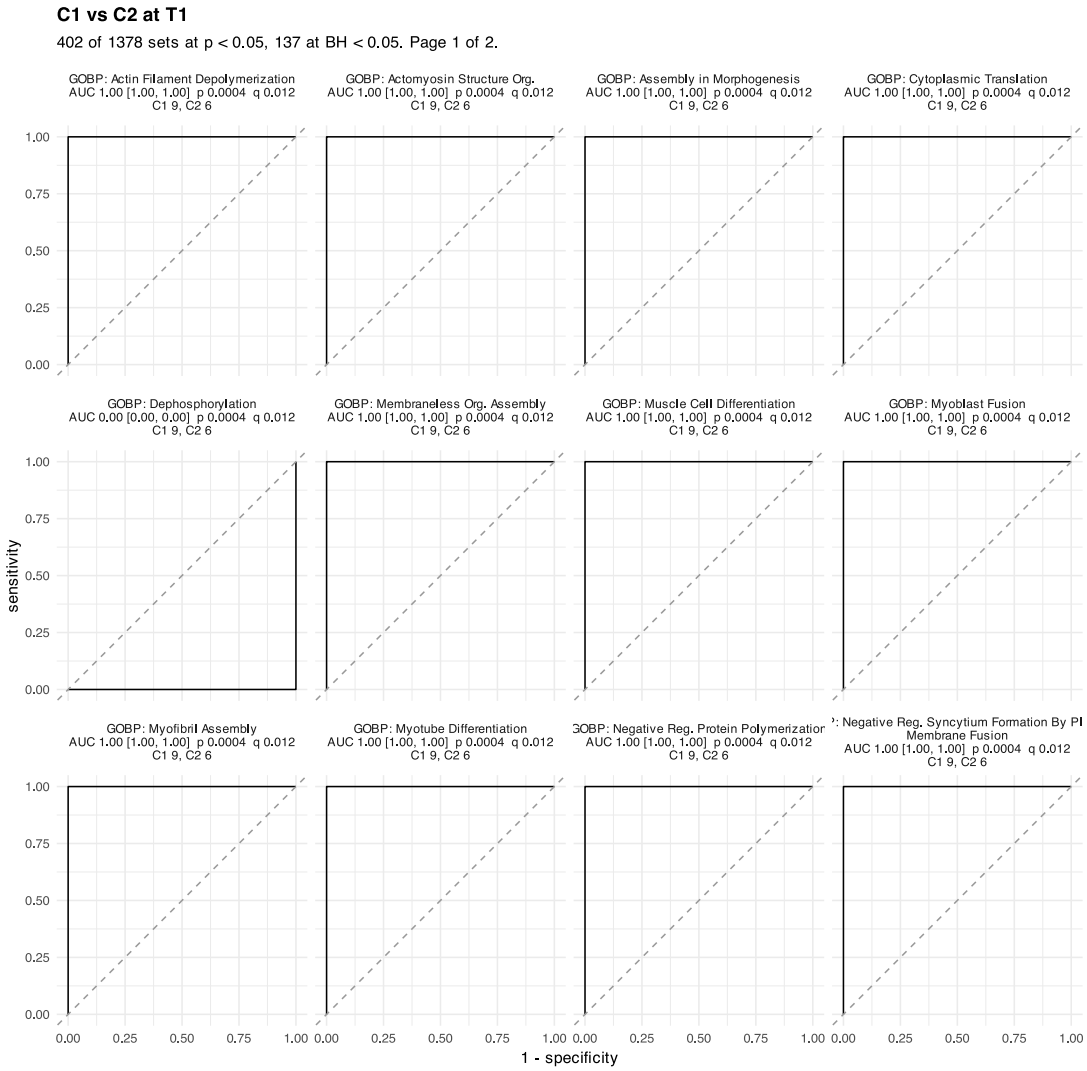


Figure 17: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

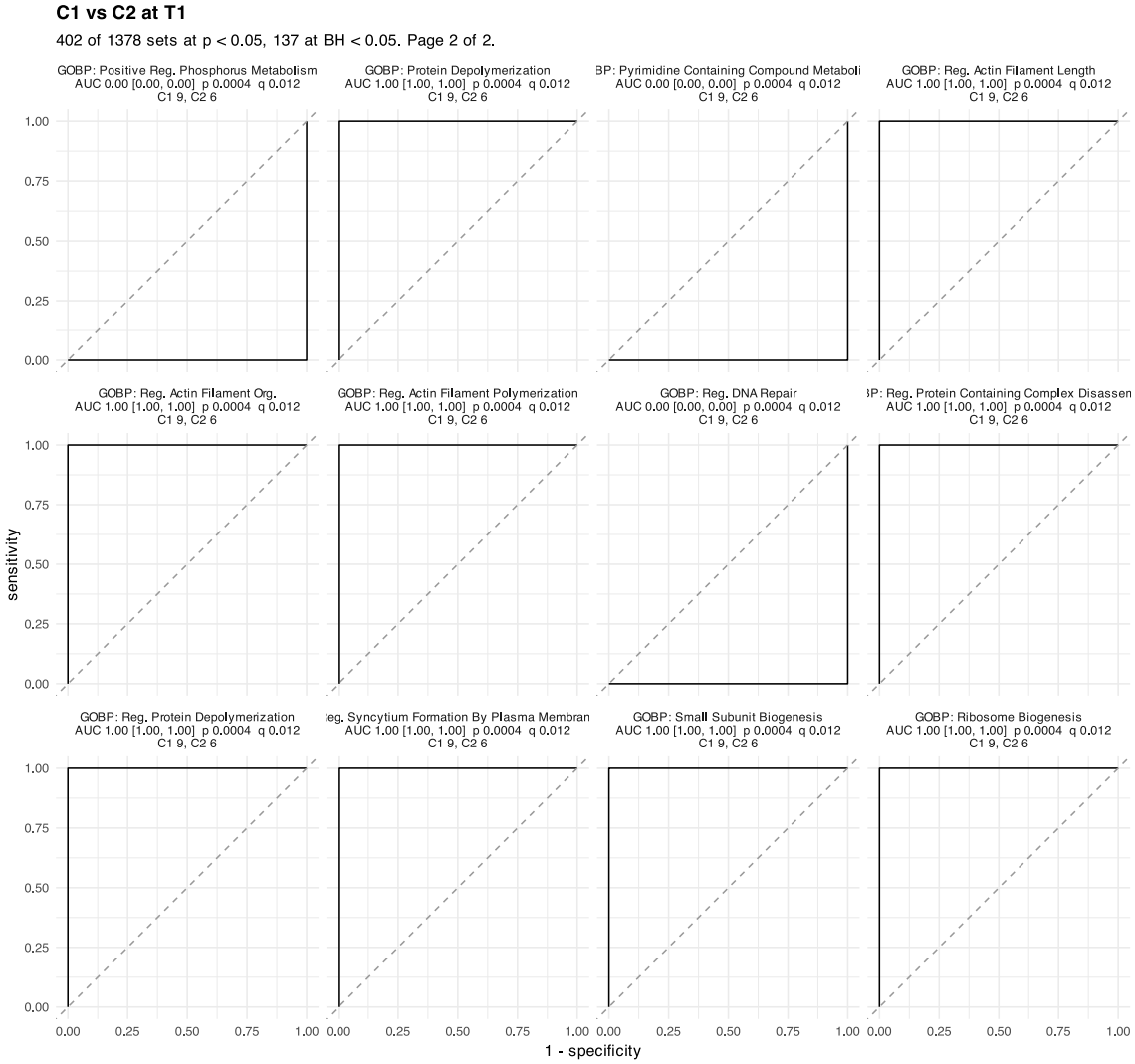


Figure 18: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

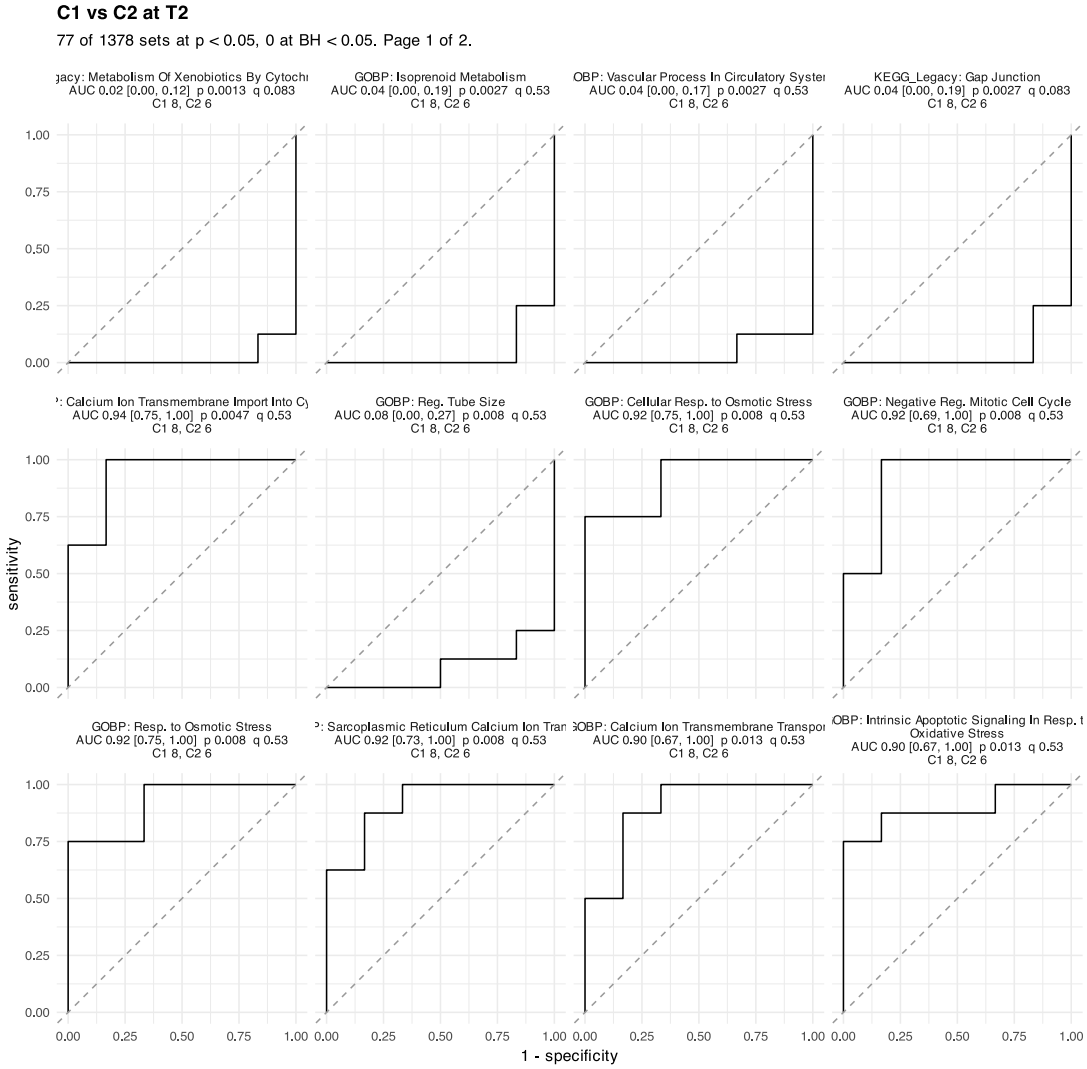


Figure 19: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

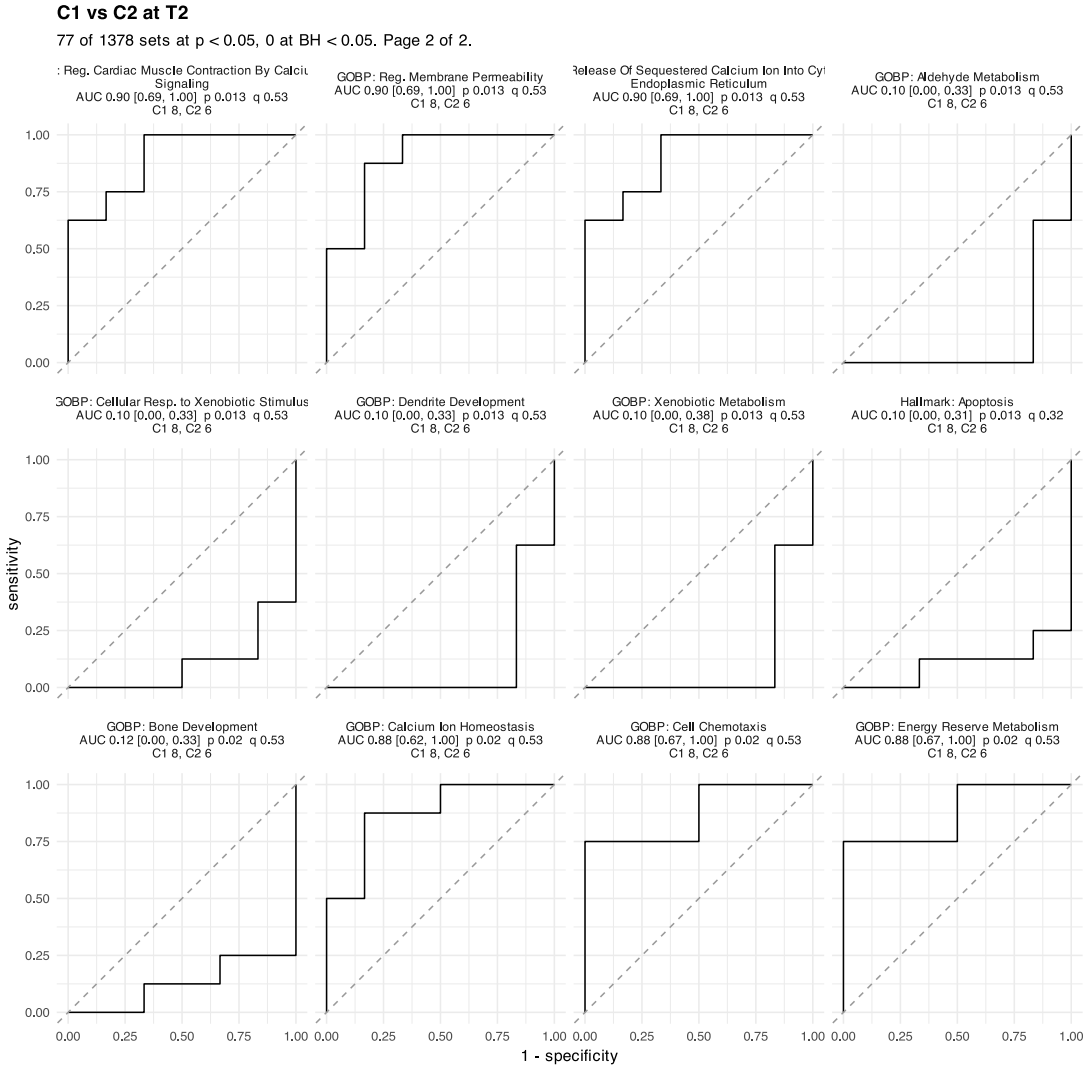


Figure 20: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

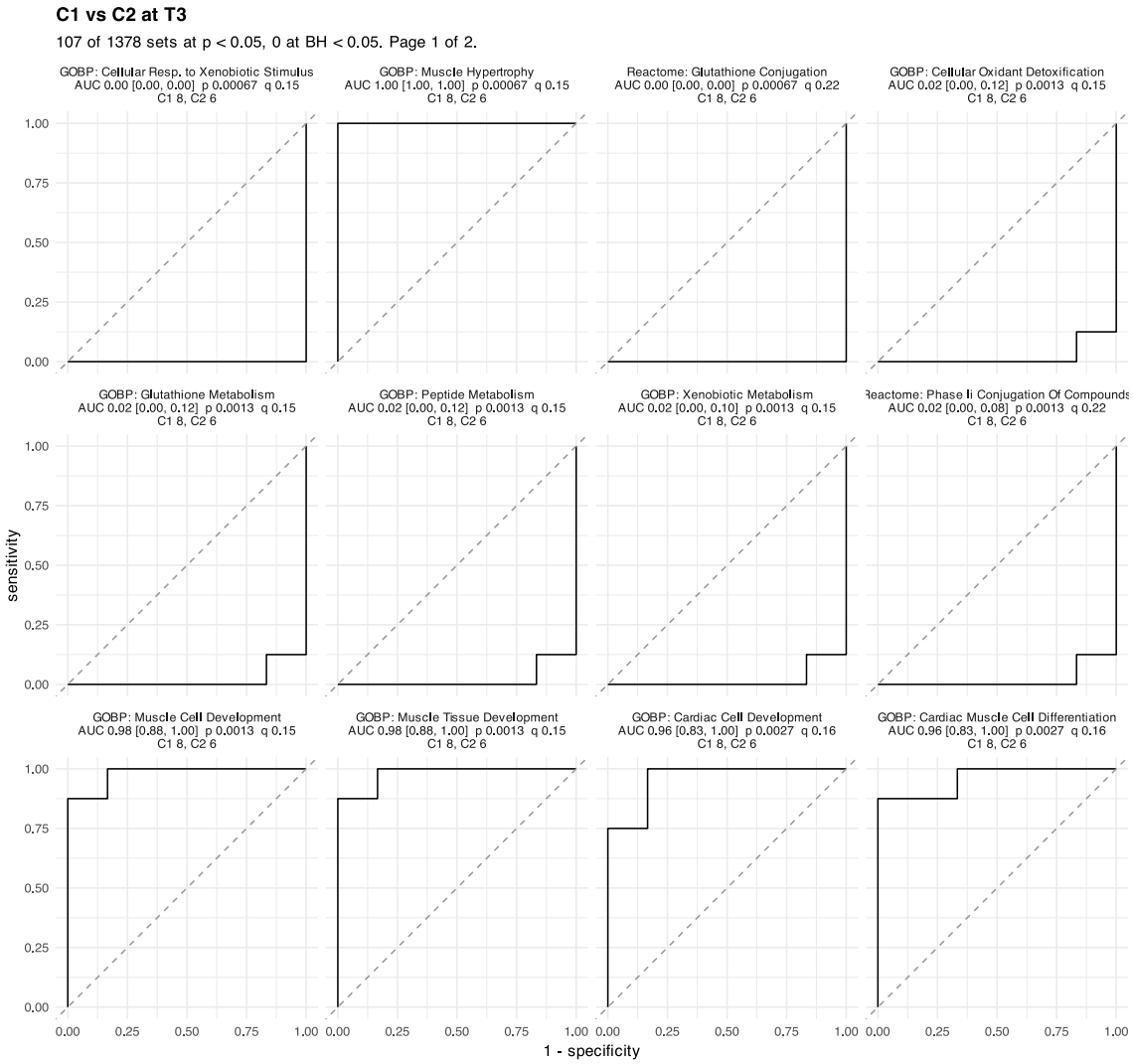


Figure 21: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

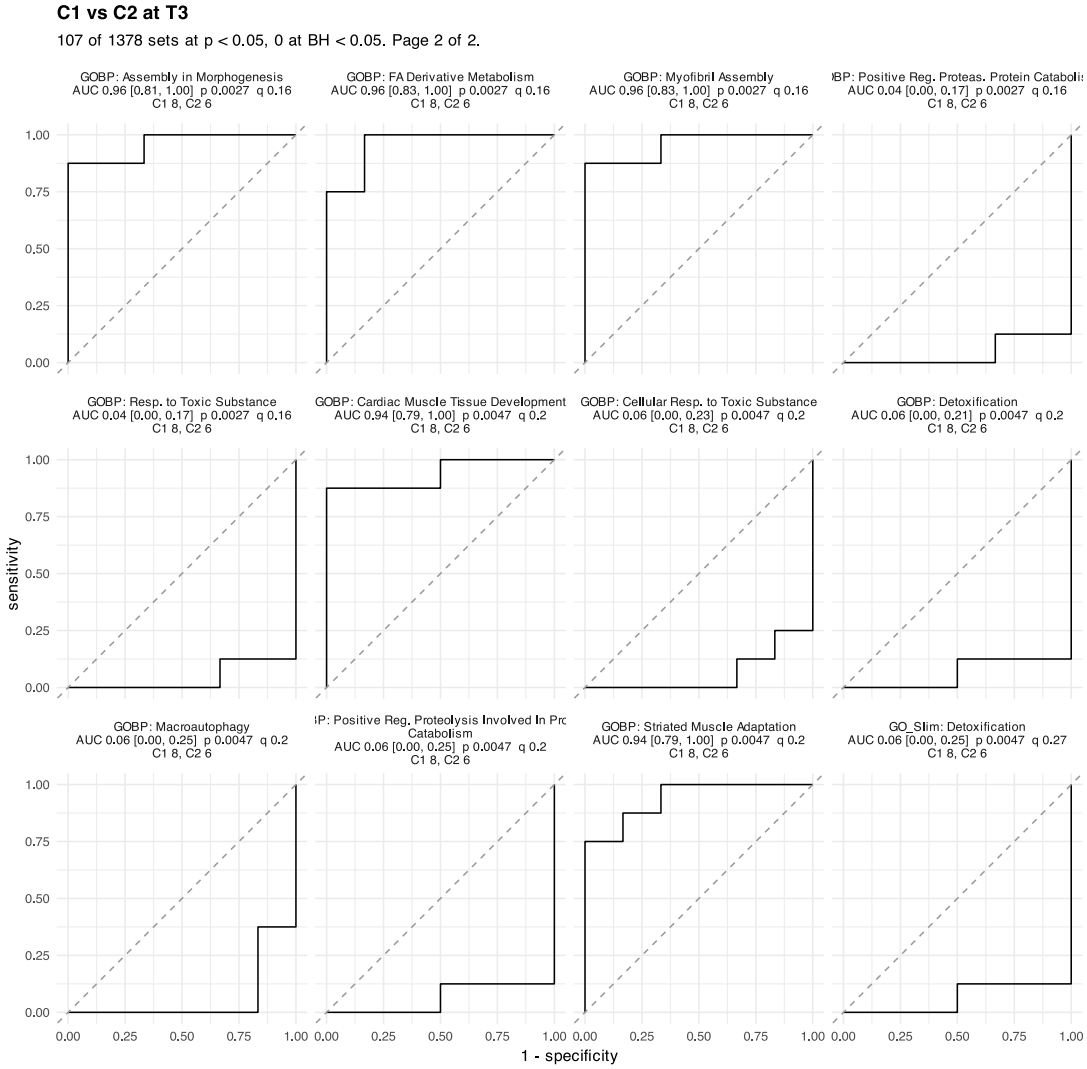


Figure 22: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

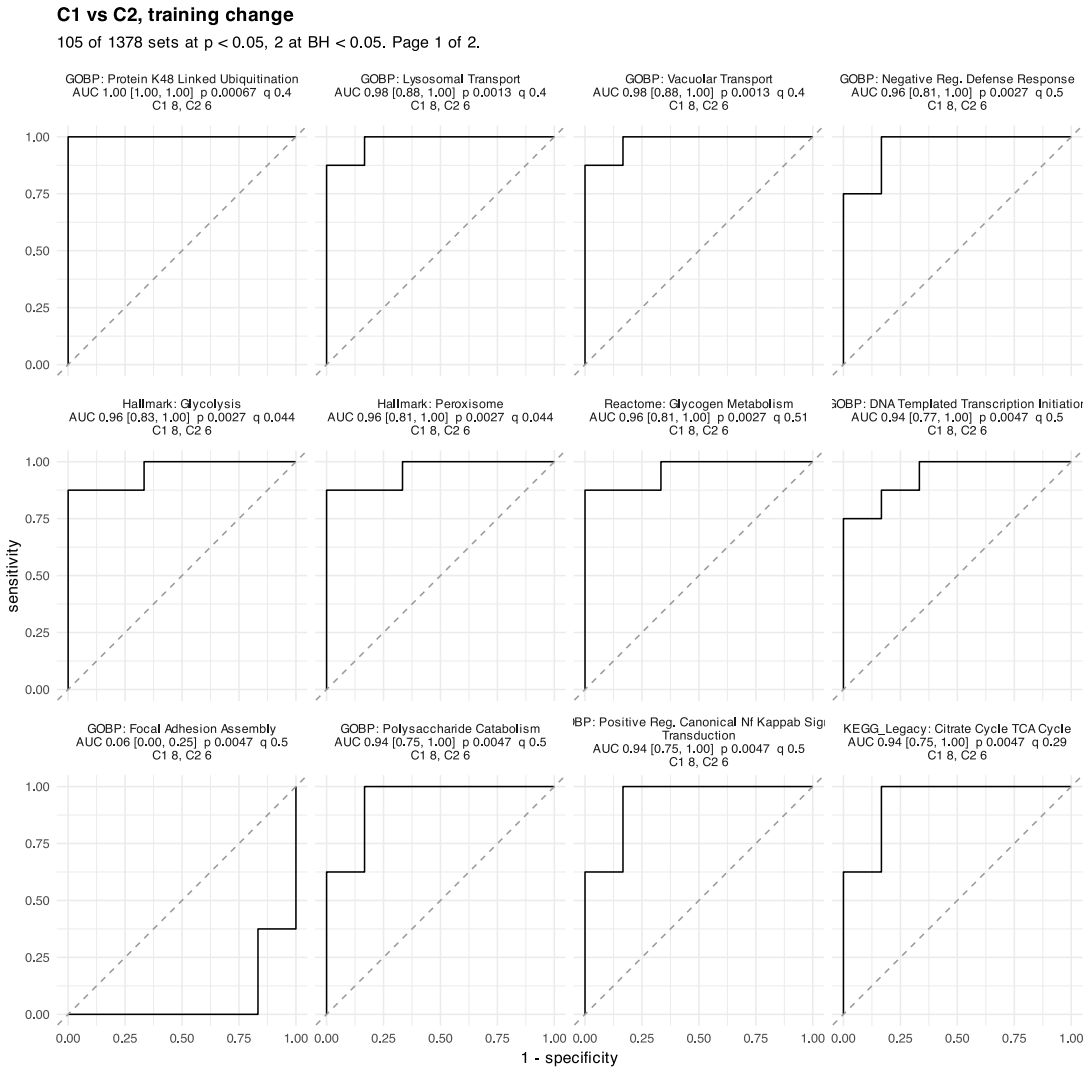


Figure 23: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

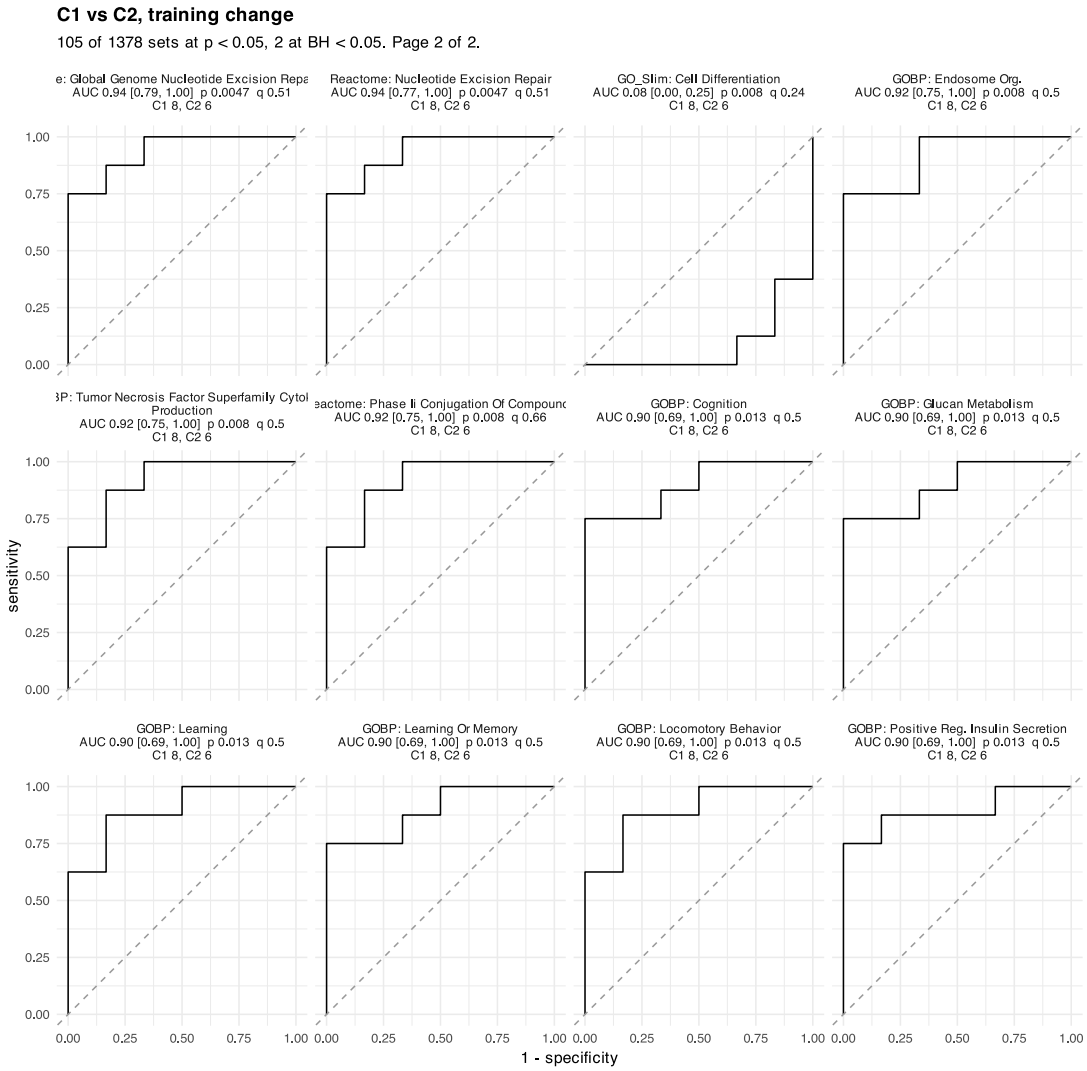


Figure 24: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

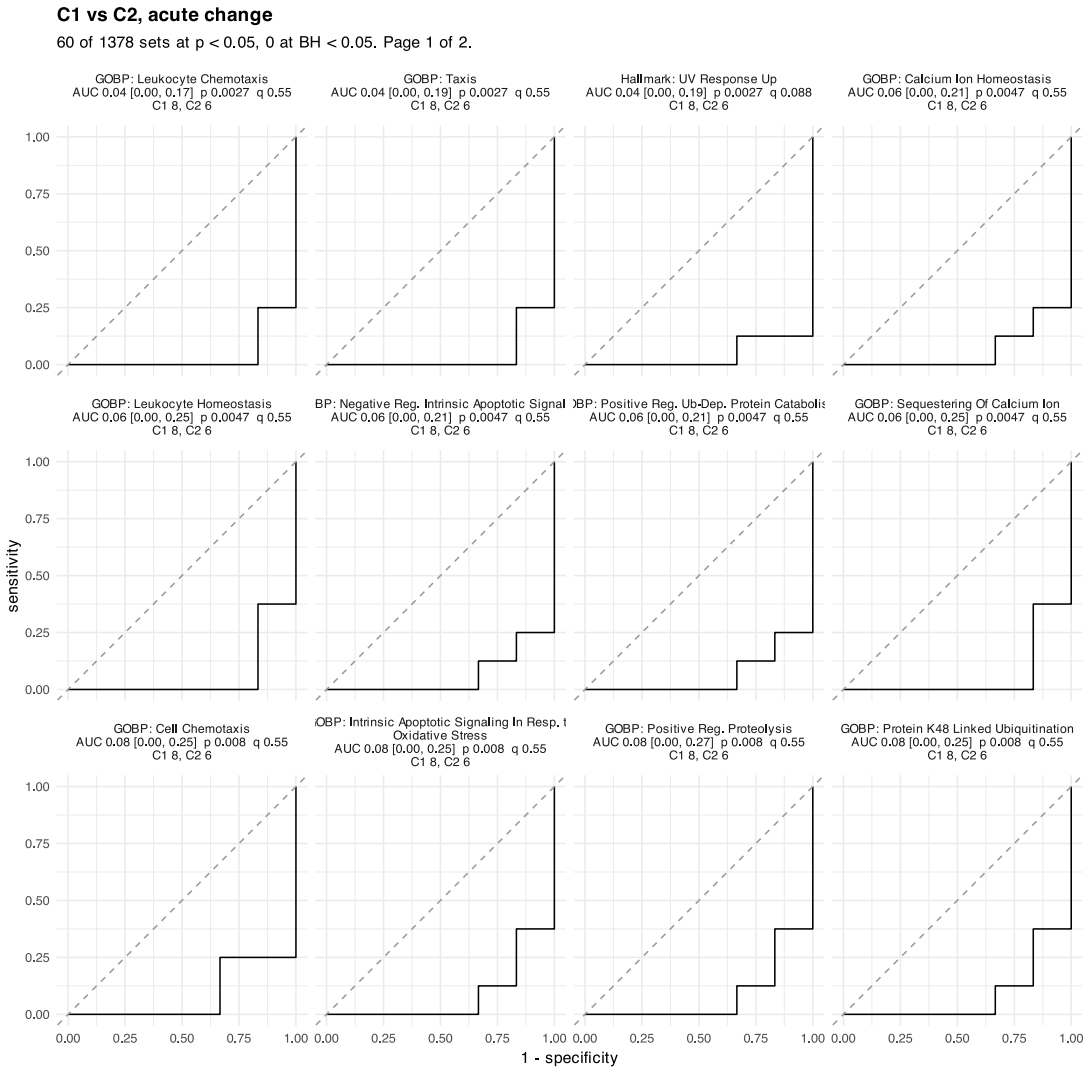


Figure 25: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

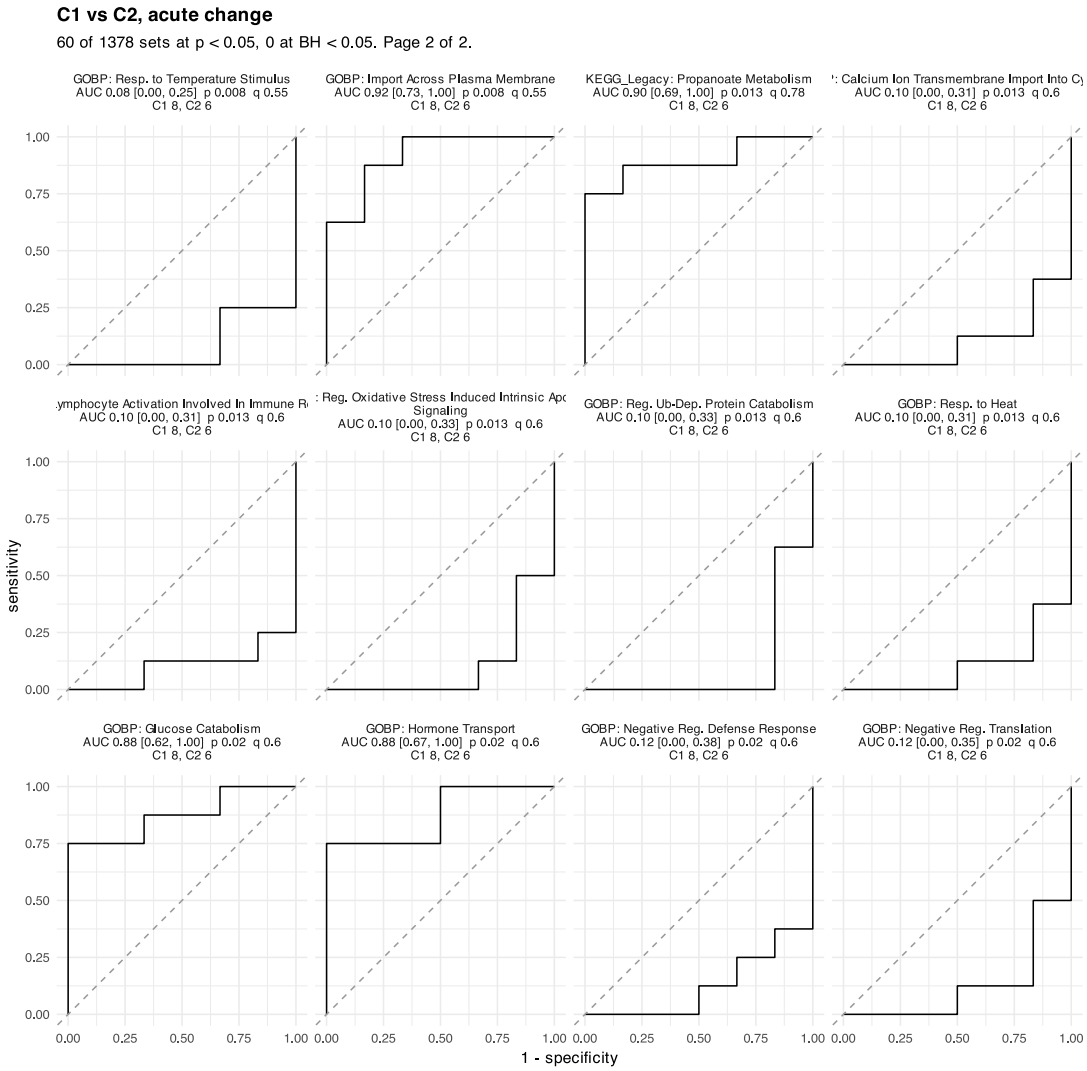


Figure 26: Gene sets: AUC and p per task and collection, then the strongest p < 0.05 sets.

Modules

Each module's eigengene per run; rows follow module size.

```
inputs <- c(design = design_file, modules = "01_Processing/06_Module_Building/c_data/
modules.rds")
paths <- map_chr(inputs, here)
stopifnot(file.exists(paths))
modules <- subset_for_design(readRDS(paths[["modules"]]), d)
stopifnot(identical(colnames(modules$eigengenes), modules$meta$sample_id))
module_counts <- classify_level(
  out = here(data_dir), name = "05_module_classification",
  inputs = inputs, paths = paths, values = modules$eigengenes, meta = modules$meta,
  feature_label = set_names(rownames(modules$eigengenes)), feature_noun = "modules",
  feature_unit = "module", row_groups = list(modules = modules$module_summary$module),
```

```
  row_order = "Rows follow module size."
)
```

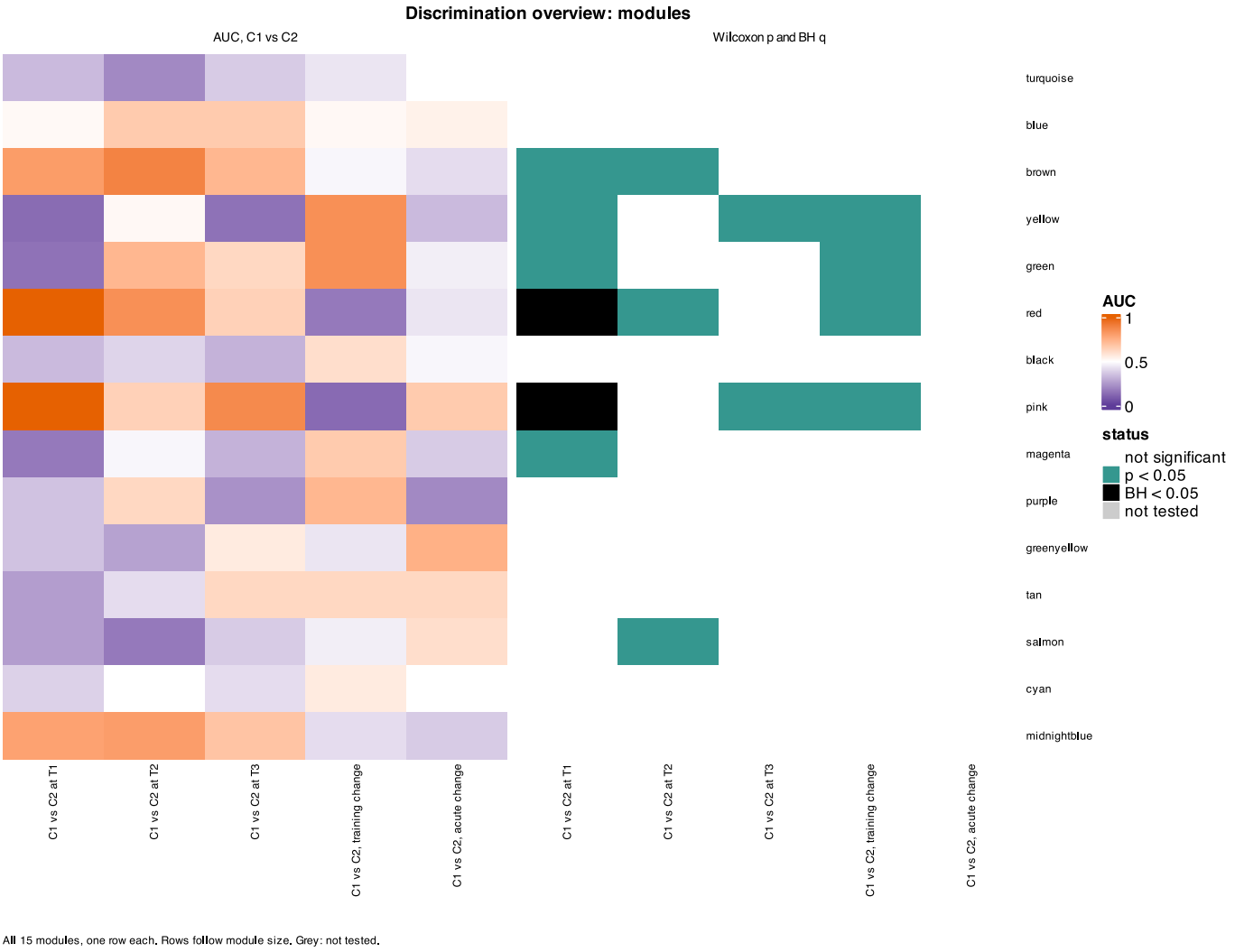


Figure 27: Modules: AUC and Wilcoxon p per task, then the strongest p < 0.05 modules.

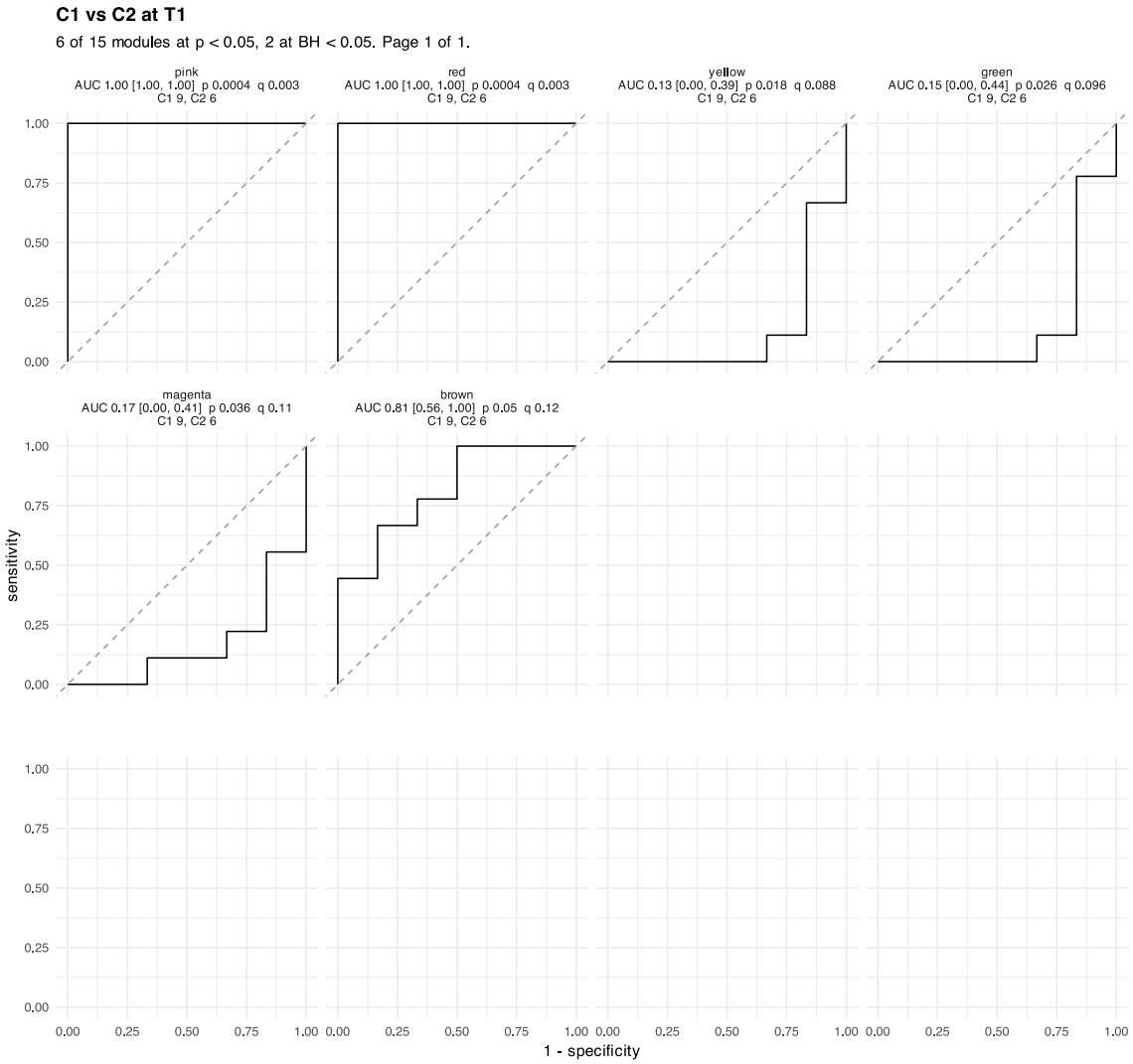


Figure 28: Modules: AUC and Wilcoxon p per task, then the strongest p < 0.05 modules.

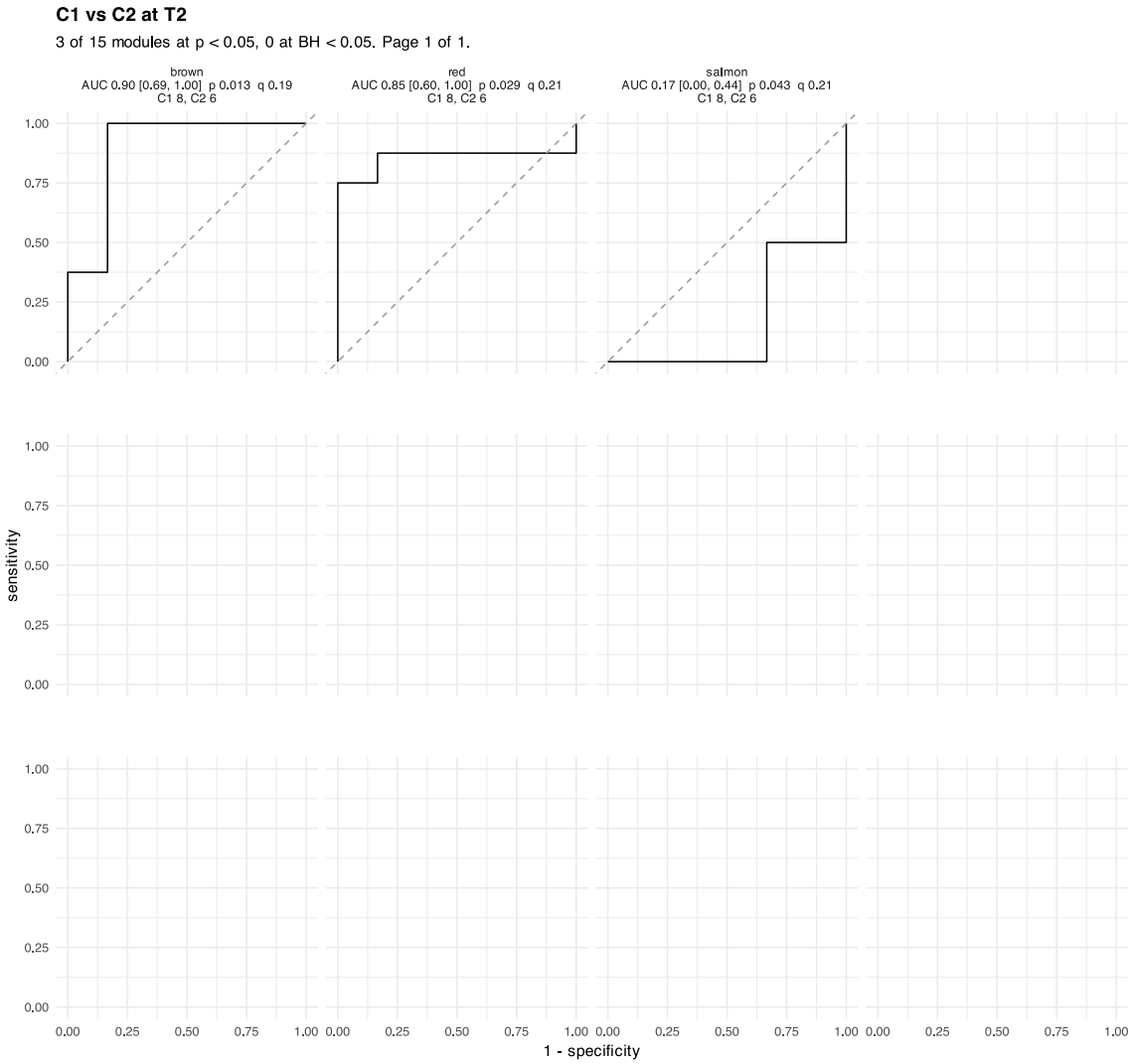


Figure 29: Modules: AUC and Wilcoxon p per task, then the strongest p < 0.05 modules.

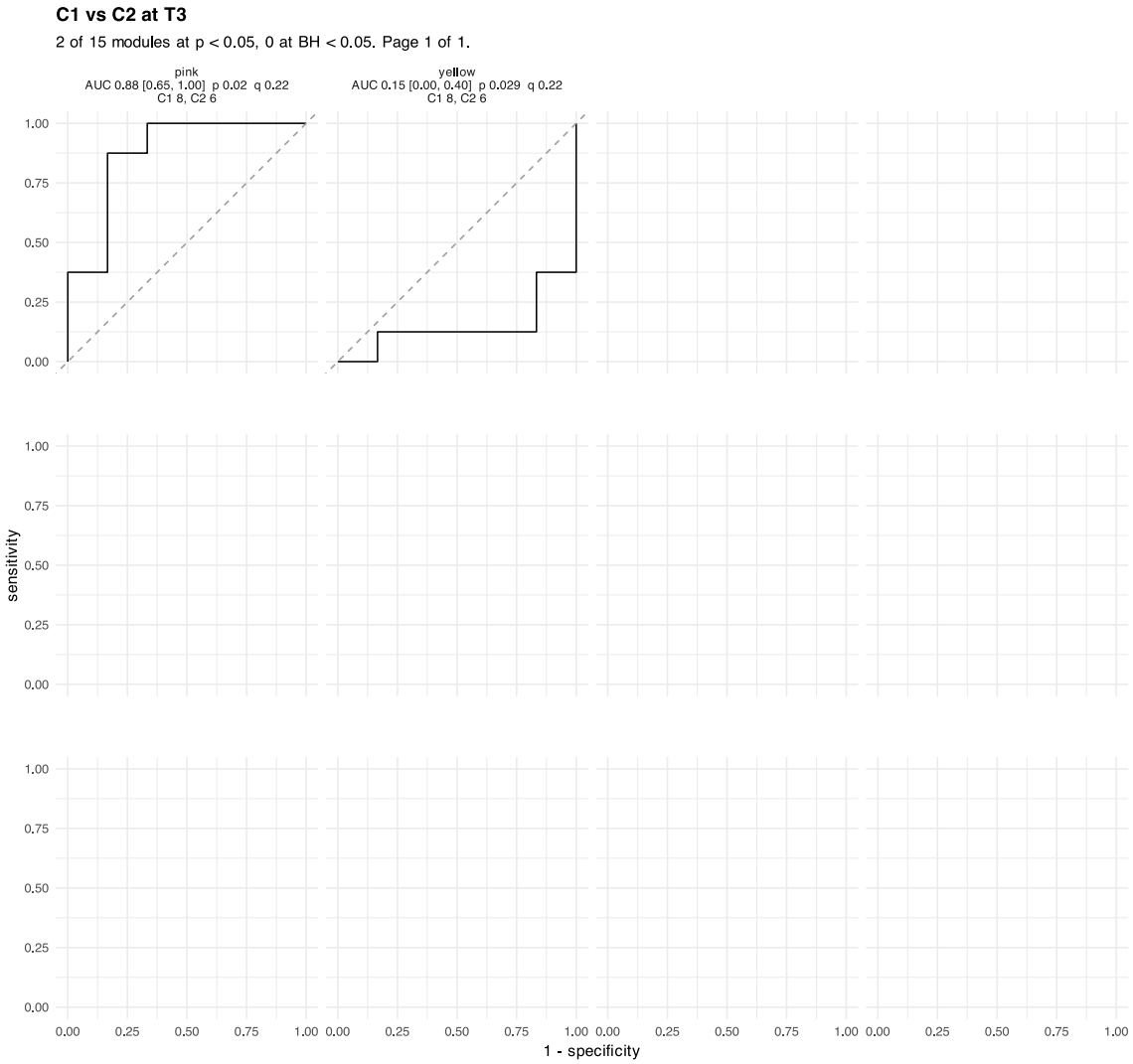


Figure 30: Modules: AUC and Wilcoxon p per task, then the strongest p < 0.05 modules.

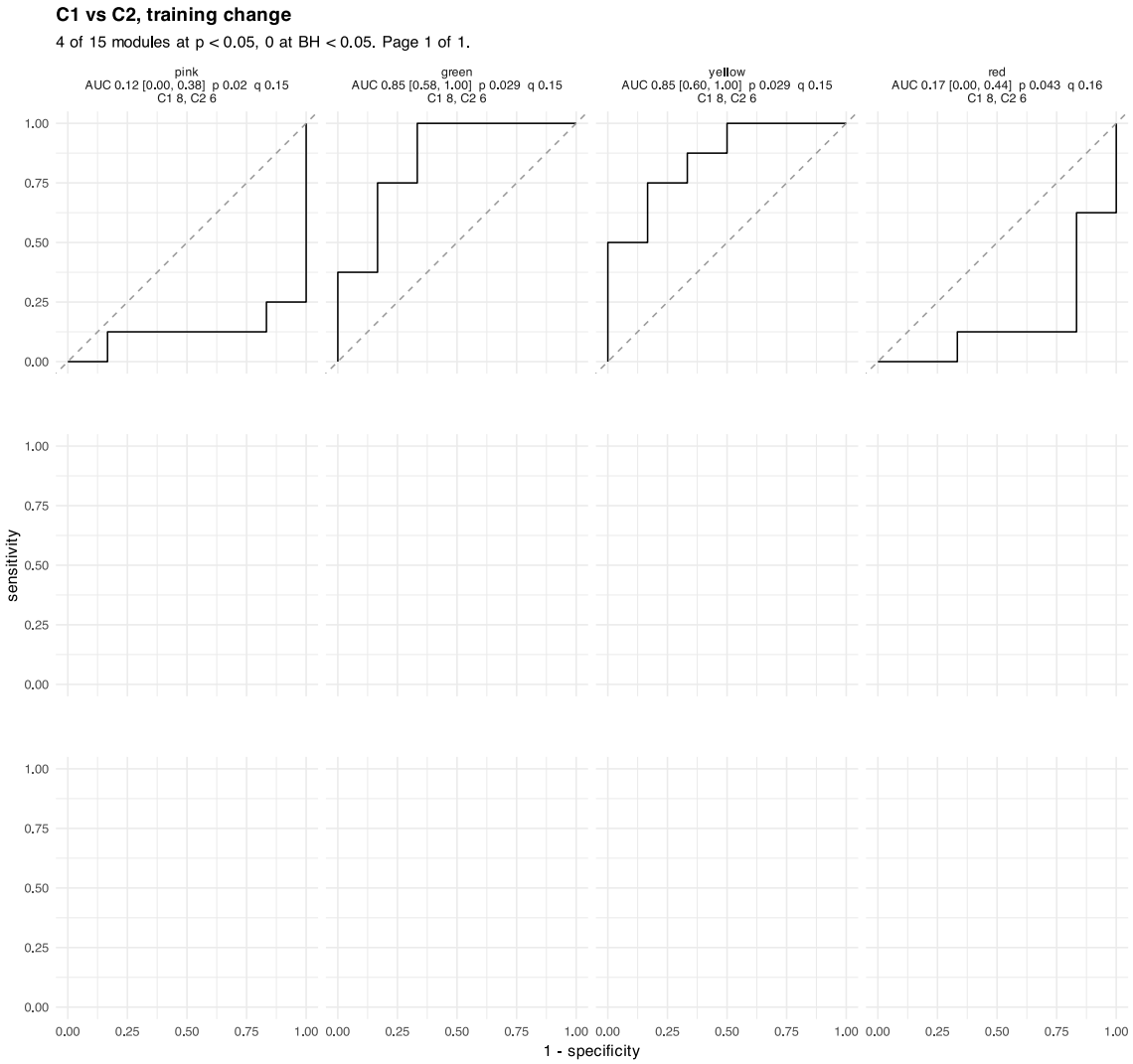


Figure 31: Modules: AUC and Wilcoxon p per task, then the strongest p < 0.05 modules.

Summary

```
bind_rows(
  proteins = summarise(protein_counts, n_nominal = sum(n_nominal), n_fdr_05 = sum(n_fdr_05)),
  sets = summarise(set_counts, n_nominal = sum(n_nominal), n_fdr_05 = sum(n_fdr_05)),
  modules = summarise(module_counts, n_nominal = sum(n_nominal), n_fdr_05 = sum(n_fdr_05)),
  .id = "level"
)
```

```
# A tibble: 3 × 3
  level    n_nominal n_fdr_05
  <chr>        <int>    <int>
1 proteins       888       64
2 sets           751      139
3 modules         15        2
```

Across the five tasks, 64 proteins, 139 set tests and 2 module tests pass BH < 0.05. With 9 against 6 people at T1 the smallest reachable p is 4^{-4}.